

Quaderno di lavoro

Che cosa è cambiato e perché

Settembre 2026 — dal modello
al datapath reale

Progetto **IPA/eBPF**

inferenza di una rete neurale dentro il kernel, a bordo del pacchetto

Ogni numero di questo quaderno viene da una misura eseguita,
non da una stima. Dove una cosa non è stata verificata, è scritto.

Indice

1	Di che cosa stiamo parlando	4
1.1	L'idea in una pagina	4
1.2	Dove gira: XDP	4
1.3	Le tre pipeline	4
2	La matematica che sta dentro il kernel	5
2.1	Perché i numeri sono interi	5
2.2	Il problema dei bias, e l'errore che ci stava dentro	5
2.3	Il TTL: dividere il prodotto, mai il valore	6
3	Le classi non sono porte	7
3.1	Il problema	7
3.2	La soluzione: una catena esplicita	7
3.3	Un errore che questa separazione ha fatto emergere	8
3.4	Ogni modello ha la sua semantica (24 settembre)	8
4	Il motore non è lo scenario	10
4.1	Com'era	10
4.2	Com'è adesso: tre strati	10
4.3	La prova	11
5	Il modello non è quello depositato	12
5.1	La domanda	12
5.2	I modelli sintetici	12
5.3	Che cosa era già provato, e che cosa no	12
5.4	Il confronto a quattro vie	13
5.5	Il risultato	13
5.6	Ma ci è arrivato trovando due difetti	14
5.7	La correzione	14
5.8	Che cosa è costata, e la prova che regge	15
5.9	Tre difetti, e chi li ha trovati	15
5.10	Che cosa questo capitolo dimostra	16
6	Dalla chiamata di sistema al kernel vero	17
6.1	Che cosa provavano i test di prima	17
6.2	Il banco nuovo	17
6.3	Tre difetti del test, trovati misurando	17
6.4	Il risultato	18
7	La feature che non c'era	19
7.1	Il sintomo	19
7.2	Perché nessun test lo vedeva	19
7.3	La correzione	19
7.4	La prova che adesso è viva	20
7.5	Il prezzo, misurato	20

8	Il nodo non sapeva di essere un nodo	21
8.1	L'ingresso più grande di tutti	21
8.2	Il sintomo	21
8.3	La correzione, e perché una tabella e non un array	21
8.4	Una pipeline ha resistito un giorno	21
8.5	La prova che il nodo è vivo	22
9	Il verificatore non conta le istruzioni	23
9.1	Due monete diverse	23
9.2	Il ciclo era girato dalla parte sbagliata	23
9.3	Perché il risultato non cambia di un bit	23
9.4	Il risultato, che sembra un errore	23
9.5	Un effetto collaterale che vale una riga di tabella	24
10	Un bivio per ogni neurone (28 settembre)	25
10.1	Il sintomo	25
10.2	Che cosa fa il verificatore	25
10.3	Dove stavano i bivi: la ReLU	25
10.4	La correzione: la ReLU senza bivio	26
10.5	Il risultato	26
10.6	Che cosa costa	27
11	I numeri, e che cosa dicono davvero	29
11.1	La tabella	29
11.2	Primo risultato: la dimensione non predice la velocità	29
11.3	Secondo risultato: quanto valgono le latenze assolute	30
11.4	Terzo risultato: la genericità ha un costo di verifica	30
11.5	Quarto risultato: togliere il compilatore dal nodo	31
11.6	Che cosa questi numeri non dicono	31
12	Come cresce il costo	32
12.1	La domanda	32
12.2	Come è impostata la misura	32
12.3	Profondità: chi paga in spazio e chi in tempo	33
12.4	Aggiornare il modello: con l'oggetto precompilato, pari	34
12.5	Larga o profonda? Dipende da chi la esegue	35
12.6	Congelare il nodo: conviene?	37
12.7	Congelare quali porte esistono	40
12.8	Due larghezze, due risposte opposte	41
12.9	I pesi contano, ma solo per una delle tre	42
12.10	Che cosa questi grafici non dicono	43
13	Il costo del programma, misurato da solo	44
13.1	La campagna sulle architetture	44
13.2	Il problema: le MAC nominali non predicono la latenza	44
13.3	La soluzione: contare le MAC che vengono eseguite	45
13.4	Due banchi, lo stesso programma, lo stesso numero	45
13.5	Il programma sul percorso vero: tre marcature	46

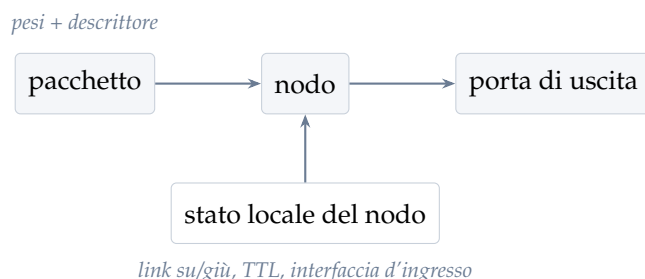
14 Il test reale: quanto costa un pacchetto al nodo	47
14.1 La domanda, in parole semplici	47
14.2 Com'è fatto il banco	47
14.3 Come si misura, e perché così	48
14.4 I due generatori	48
14.5 Com'era il test nella macchina virtuale, e com'è adesso	49
14.6 I limiti del test, con esempi	50
14.7 Il risultato: quanto regge il nodo	52
14.8 Il costo del nodo, con pacchetti grezzi	53
14.9 Al crescere del bit rate	53
14.10 Che cosa questo numero non dice	54
15 La macchina di laboratorio, sotto controllo (27 settembre)	55
15.1 Un processore che non sta fermo	55
15.2 Le condizioni, messe e tolte	55
15.3 La frequenza scritta non è la frequenza vera	56
15.4 Quale frequenza	56
15.5 Il risultato: misure che si ripetono	57
15.6 Prima e dopo: che cosa è cambiato	57
15.7 Che cosa ha portato alla luce	58
15.8 Come si rifà tutto	58
16 Le altre correzioni di settembre	59
16.1 Il P1 che va sui nodi non inoltrava	59
16.2 Tutti i test di P1 usano l'oggetto AOT	59
16.3 Il cronometro del caricatore AOT	59
16.4 Due implementazioni che devono dare lo stesso numero	60
16.5 La scala delle feature in P2 e P3	60
16.6 Pulizia del repository	60
17 I numeri di oggi, in una pagina	61
18 Quel che resta	62
18.1 Fatto	62
18.2 Aperto	62
18.3 Il filo conduttore	63
19 I comandi	64
19.1 La macchina	64
19.2 Correttezza: il modello fa quello che deve	64
19.3 Kernel: si carica, e quanto costa	64
19.4 Datapath reale: il pacchetto attraversa davvero	65
19.5 Analisi parametrica: come cresce il costo	65
19.6 Il test reale	65
19.7 Costi specifici e oggetto precompilato	66
19.8 L'ordine consigliato	66

Di che cosa stiamo parlando

1.1 L'idea in una pagina

Un router normale decide dove mandare un pacchetto consultando una tabella riempita da un protocollo di routing. L'idea di **IPA** (*Intelligent PAckets*) è diversa: il pacchetto **porta con sé** una piccola rete neurale, e ogni nodo che attraversa la esegue per decidere l'uscita.

Niente segnalazione, niente tabella condivisa: la decisione nasce dai dati che il nodo ha sotto mano in quel momento.



Il modello è una rete piccola: quattro tipi di ingresso, due strati nascosti da quattro neuroni, sette uscite. In tutto **319 pesi**. Deve stare in un header ed essere eseguibile in decine di nanosecondi.

1.2 Dove gira: XDP

Il codice gira in **eBPF** agganciato a **XDP**, il punto più precoce in cui il kernel Linux permette di toccare un pacchetto: prima che venga costruita la struttura `sk_buff`, quindi prima di quasi tutto il resto dello stack di rete.

Questo impone un vincolo forte, che spiega metà delle scelte di progetto: un programma eBPF deve essere **dimostrato sicuro** da un componente del kernel chiamato *verificatore* prima di poter girare. Niente cicli di lunghezza ignota, niente accessi non controllati alla memoria, niente ricorsione. Tutto ciò che nel documento sembra contorto — cicli srotolati, tetti dichiarati a priori, tabelle al posto di calcoli — nasce da lì.

1.3 Le tre pipeline

Lo stesso modello è implementato in tre modi, per misurare un compromesso.

	i pesi stanno	cambiare modello
P1 hardcoded	scritti nel codice C	ricompilare
P2 template	in una mappa	scrivere la mappa
P3 modular	in una mappa, uno strato per programma	scrivere la mappa

P1 è veloce perché il compilatore vede i pesi e li semplifica; è rigido perché ogni modello nuovo richiede una ricompilazione. P2 e P3 sono l'opposto. Il lavoro di settembre serve a rendere questo confronto *misurabile davvero*.

La matematica che sta dentro il kernel

2.1 Perché i numeri sono interi

Nel kernel non ci sono numeri in virgola mobile: eBPF lavora con interi. Il modello però è addestrato in virgola mobile. Serve una conversione, e si chiama **quantizzazione**.

Ogni peso reale w diventa un intero a 8 bit:

$$w_{\text{int}} = \text{round}(w \cdot s)$$

dove s è il **fattore di scala**, uno solo per tutta la rete.

Perché il risultato stia in un byte con segno serve

$$s \cdot \max_i |w_i| \leq 127.$$

Sul modello depositato $\max |w| = 5,098$, quindi $s \leq 24,9$. Due scelte convivono nel progetto: $s = 24$ (il massimo intero ammesso) e $s = 16$ (la potenza di due più vicina per difetto, che rende le divisioni degli shift).

Superare quel limite non dà un errore: dà un *overflow silenzioso*. I pesi grandi si troncano e il modello sbaglia senza che nulla lo segnali. Per questo i test distinguono le scale *rappresentabili* da quelle che saturano, e misurano solo sulle prime.

2.2 Il problema dei bias, e l'errore che ci stava dentro

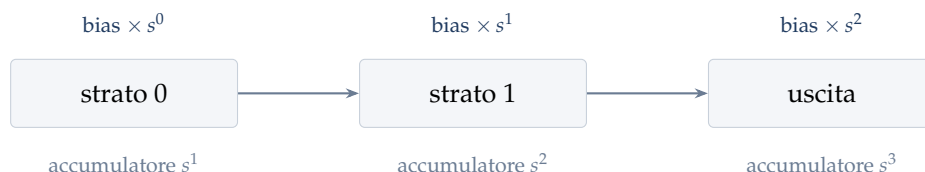
Qui sta l'errore più sottile trovato in settembre, e vale la pena seguirlo passo passo perché è istruttivo.

Quando moltiplichiamo un ingresso per un peso quantizzato, il risultato porta un fattore s in più rispetto al valore reale:

$$x \cdot w_{\text{int}} = x \cdot w \cdot s.$$

Se impiliamo gli strati, quel fattore si accumula. Dopo $\ell + 1$ moltiplicazioni l'accumulatore dello strato ℓ porta $s^{\ell+1}$.

Ma il **bias** è memorizzato come un peso qualsiasi, cioè con un solo fattore s . Sommarlo così com'è significa sommare una quantità sistematicamente troppo piccola. Va riportato nelle unità dell'accumulatore:



La regola è semplice: allo strato ℓ (contando da zero) il bias va moltiplicato per s^ℓ .

L'errore

Il modello di riferimento in Python — quello con cui si controlla che il datapath calcoli la cosa giusta — applicava s^0 e s^1 correttamente ai primi due strati, e lasciava l'**ultimo strato** a s^0 invece di s^2 .

Il bias dell'uscita era sottopesato di $s^2 = 24^2 = 576$.

Perché nessuno se n'era accorto per mesi

Tutti i test eseguivano il modello con **tutti i link attivi**. In quella condizione gli accumulatori sono grandi e un errore di 576 non sposta mai il massimo. Basta spegnere qualche link perché gli accumulatori si riducano e l'errore decida l'esito.

Nello stesso file c'era una seconda funzione di riferimento, scritta per forme arbitrarie, che usava la formula generale s^ℓ ed era **già corretta**. Due riferimenti, uno giusto e uno sbagliato, e quello sbagliato era l'unico mai esercitato con link spenti.

Corretto l'errore, il riferimento e le tre pipeline concordano su tutti i casi provati.

La lezione non è "c'era un bug". È che **un test che varia un solo ingresso non è un test**: verifica una condizione e la ripete. Ci torneremo nel capitolo 5, dove lo stesso schema si ripresenta in una forma ancora più netta.

2.3 Il TTL: dividere il prodotto, mai il valore

Il modello è addestrato con il TTL *normalizzato*, cioè ttl/T con $T = 30$ il valore iniziale. Con i numeri interi

$$\frac{\text{ttl}}{30} = 0 \quad \text{per ogni } \text{ttl} < 30,$$

e la feature sparirebbe. La divisione va quindi applicata al *prodotto*:

$$\left\lfloor \frac{\text{ttl} \cdot w_{\text{int}}}{T} \right\rfloor \quad \text{e non} \quad \left\lfloor \frac{\text{ttl}}{T} \right\rfloor \cdot w_{\text{int}}.$$

Nel C generato si legge esattamente così:

```
(((__s64)_ttl * 11LL) / 30LL)
```

Le classi non sono porte

3.1 Il problema

La rete produce sette numeri; il più grande vince. Ma *che cosa significa* la classe vincente?

All'inizio di settembre la risposta era sparsa e incoerente. In punti diversi del progetto convivevano quattro convenzioni su quale classe significasse "scarta il pacchetto":

Convenzione	Dove
la classe 6 è DROP	in un punto
la classe 5 è DROP	in un altro
$\text{DROP} = n_{\text{out}} - 1$	in un terzo
$\text{DROP} = \max(\text{etichette}) + 1$	dedotto dai dati

E sotto ce n'era una quinta, implicita e più insidiosa: che la classe k volesse dire "manda fuori dall'interfaccia k ". Funzionava per caso, perché nel modello depositato le porte sono numerate come le classi.

3.2 La soluzione: una catena esplicita

Adesso il significato è **dichiarato**, mai dedotto, e passa per una catena di quattro passi, ognuno con un responsabile diverso:



- **classe**: quello che il modello ha calcolato;
- **azione**: FORWARD, DROP o UNUSED — una classe mai addestrata esiste ed è dichiarata tale;
- **porta logica**: un numero che il *modello* conosce;
- **ifindex**: l'interfaccia vera, che solo il *nodo* conosce e che è diversa su ogni macchina.

Nel modello depositato la dichiarazione è:

classe	azione	porta logica
0-4	FORWARD	0-4
5	DROP	—
6	UNUSED	—

Il punto non è che adesso funziona: funzionava anche prima, per coincidenza numerica. Il punto è che adesso **smetterebbe di funzionare rumorosamente** se il modello cambiasse, invece di continuare a girare dando risposte plausibili e sbagliate.

3.3 Un errore che questa separazione ha fatto emergere

Il datapath teneva un contatore per classe, ma scriveva nella casella della *porta* invece che in quella della *classe*. Sembrava giusto perché le due numerazioni coincidono nel modello depositato.

Con un modello in cui non coincidono, il contatore avrebbe misurato la cosa sbagliata in silenzio. Separare i due spazi di indici ha reso l'errore visibile e correggibile.

3.4 Ogni modello ha la sua semantica (24 settembre)

Il limite. In P2 e P3 la tabella classe $\rightarrow$ azione era *una*, letta con la sola classe e condivisa da tutti i modelli caricati. Due modelli non potevano dare alla stessa classe significati diversi: per evitare che il secondo riscrisse in silenzio le azioni del primo, il piano di controllo lo rifiutava. Il vantaggio di P2 e P3 — un programma solo, i modelli caricati come dati — valeva quindi solo per modelli con la stessa semantica.

La soluzione. La chiave della tabella è diventata la coppia (model_id, classe). Il programma resta uno per architettura; aggiungere un modello resta scrivere delle tabelle.



- **Una tabella ad accesso diretto, non una tabella hash:** 256 modelli $\times$ 2 banchi $\times$ 32 classi. Resta una sola lettura per pacchetto, come prima, e nessun ciclo sui modelli registrati.
- **Due banchi per modello.** Ricaricare un modello scrive il banco inattivo, poi un'unica scrittura del registro cambia insieme il numero di classi e il banco in uso. Un pacchetto vede la semantica vecchia o quella nuova, mai una tabella scritta a metà.
- **Perché non dentro il registro,** che il programma ha già letto: usare la classe come indice dentro quel valore costringe il verificatore a seguirne il valore esatto attraverso tutto l'argmax, e smette di potare i cammini. Misurato: layer_hidden di P3 non si carica più.
- **Senza un valore vivo in più.** Il primo strato di P3 sta al limite dei 512 byte di stack nelle sue versioni strumentate (quella che conta le letture e quella con i timestamp T1/T2 del test reale): tenere vivi fino alla fine model_id e il registro la faceva sforare. La chiave viaggia allora nella variabile che già diceva «questo è l'ultimo strato», e il programma cresce di 9 istruzioni.

La prova. Modelli costruiti in modo che il TTL scelga la classe, così ogni classe di ogni modello è raggiungibile: un modello solo, due con la stessa semantica, due con semantiche opposte su ogni classe a pacchetti alternati, un modello inesistente o rimosso, numeri di classi diversi, un modello ricaricato con un'altra semantica. **53/53** su P2 e P3. Con una tabella condivisa simulata gli stessi test falliscono dove devono: il test distingue le due cose.

	P2		P3	
	classe sola	per modello	classe sola	per modello
istruzioni	14 978	14 969	12 277	12 318
letture di tabella	11	11	29	29
latenza, minimo	192 ns	191 ns	325 ns	319 ns
memoria delle tabelle	11 192 B	142 264 B	41 020 B	172 092 B

Il datapath non se ne accorge: stesse letture, poche istruzioni in più o in meno, latenza entro il 2%, e nessuna crescita da 1 a 4 modelli (a 8, P3 sale a 340 ns: un punto solo, non spiegato). Il prezzo è in memoria: 128 KiB fissi per pipeline, riservati per 256 modelli anche quando ne è caricato uno solo.

I numeri in tabella sono del 27 settembre. Il 28 la misura è stata rifatta dopo la ReLU senza bivio (capitolo 10), ma il confronto è con l'albero di prima della semantica per modello, che ha ancora la ReLU vecchia: la differenza mescolerebbe due modifiche, e la tabella resta quella pulita. Una cosa però il 28 l'ha detta: il punto a 8 modelli di P3 **non si è ripetuto** (315 ns, come con un modello solo). Era rumore.

Il motore non è lo scenario

4.1 Com'era

Il progetto era nato attorno a una topologia precisa: **Germany50**, una rete di 50 router usata come riferimento in letteratura, con due host attaccati.

Due numeri erano ovunque nel codice:

$$n_{\text{nodi}} = 52, \quad n_{\text{interfacce}} = 6.$$

Il primo è $50 + 2$. Il secondo è il **grado massimo** della rete: il nodo con più vicini. Non è il grado di un nodo qualsiasi — tutti gli altri riempiono di zeri gli slot che non usano.

Questi numeri determinano la larghezza dell'ingresso:

$$n_{\text{in}} = \underbrace{6}_{\text{link}} + \underbrace{6}_{\text{iface}} + \underbrace{1}_{\text{ttl}} + \underbrace{52}_{\text{nodo}} = 65.$$

Il problema non era che esistessero, ma che il motore li conoscesse. C'era un *default* interno: se nessuno diceva niente, il motore assumeva 6 e 52. Ogni chiamante che dimenticava di specificare lo scenario risultava silenziosamente corretto per Germany50 e silenziosamente sbagliato altrove.

4.2 Com'è adesso: tre strati



La dipendenza punta **solo verso l'alto**. Il motore non sa che cosa sia Germany50.

Le dimensioni si risolvono in ordine: una variabile d'ambiente, poi un file di sistema, poi il blocco `trained_on` del descrittore del modello. Se nessuna delle tre risponde, il motore **si ferma** e dice dove ha cercato.

Un default comodo è il modo più efficace di nascondere un'assunzione. Preferire un errore esplicito a un valore plausibile è una scelta di progetto, non pignoleria: il valore plausibile non si manifesta mai come errore, solo come risultato sbagliato mesi dopo.

4.3 La prova

Non basta dirlo. Il motore genera oggi un datapath corretto per cinque topologie che non hanno niente a che vedere con quella d'origine:

iface	nodì	strati	n_{in}	n_{out}	vettore link
3	8	4-4	15	4	v[3]
4	16	6	25	5	v[4]
5	24	4-4-4	35	6	v[5]
2	6	3-3	11	3	v[2]
6	52	4-4	65	7	v[6]

Nessuna riga di C scritta a mano cambia fra una riga e l'altra.

Il modello non è quello depositato

5.1 La domanda

Il capitolo precedente ha tolto la topologia dalle assunzioni del codice. Restava dentro un'assunzione più silenziosa: **un solo modello**. Ogni misura, ogni confronto fra pipeline e ogni prova di correttezza girava sullo stesso checkpoint fornito dal relatore — 65-4-4-7, scala 25.

Il rischio non è teorico. Un datapath che calcola la cosa giusta su un modello può calcolarla per caso: perché quel modello ha una forma particolare, perché i suoi pesi cadono in un intervallo comodo, o perché una costante scritta nel codice coincide con un suo parametro. Finché il modello è uno solo, la differenza fra “corretto” e “corretto qui” non si vede.

5.2 I modelli sintetici

La risposta sono **sette scenari generati**, con pesi casuali e nessun rapporto con il checkpoint depositato. Coprono un asse ciascuno:

Scenario	Forma	Pesi
ones	11-2-3	33
small	15-4-4	84
mixed	18-6-5	149
ipa_like	65-4-4-7	319
sparse	65-4-4-7	319
deep	65-4-4-4-7	339
large	117-8-8-9	1 097

Il generatore non produce solo numeri a caso: rispetta la **struttura** delle feature, che è la parte che un datapath può sbagliare. Ogni feature dichiara il suo tipo — `real` (un valore in un intervallo, che il datapath porta come intero diviso per una scala), `integer`, `binary` (un vettore di 0/1), `onehot` (esattamente un 1) — con i suoi estremi, la sua scala e i suoi vincoli. Ogni scenario porta con sé **1 000 vettori d'ingresso**, nella versione intera e in quella a virgola mobile, e le predizioni attese per entrambe.

Gli artefatti non stanno in git: sono rigenerabili byte per byte a parità di seme, e il test li rigenera da solo se non li trova.

5.3 Che cosa era già provato, e che cosa no

Su quei sette scenari il progetto verificava che i pesi **compilassero** e passassero il verificatore del kernel, e che i control plane accettassero i blocchi. È una prova reale ma di una cosa sola: che il programma esiste. Non dice *quale classe sceglie*.

La distinzione che mancava

Un programma che compila e gira può comunque decidere diversamente dal modello che dovrebbe eseguire. La domanda scoperta era: *a parità di ingresso, il datapath sceglie la stessa classe del riferimento?*

5.4 Il confronto a quattro vie

Due confronti non bastano, perché un disaccordo ha più di una causa possibile e le cause chiedono rimedi diversi. Le vie sono quattro:

Confronto	Che cosa isola
float vs int8 (Python)	l'errore di quantizzazione : il modello a interi decide diversamente da quello a virgola mobile. Può legittimamente non essere 100%.
int8 (Python) vs int8 (riferimento)	controllo : due implementazioni intere indipendenti del progetto devono concordare fra loro, o non è chiaro quale sia il riferimento.
int8 (riferimento) vs int8 (eBPF)	se il datapath è esatto rispetto alla configurazione che gli è stata data .
int8 (Python) vs int8 (eBPF)	l'errore di implementazione . Deve valere 100%: un solo disaccordo qui è un difetto del datapath.

Perché non i 1 000 vettori depositati

Il datapath non accetta un vettore d'ingresso arbitrario: lo *costruisce* da ciò che ha sotto mano — le mappe dense, il TTL del pacchetto, la porta d'ingresso risolta, l'indice del nodo. Un one-hot node diverso a ogni campione vorrebbe dire ricompilare il programma a ogni campione.

Si campiona quindi nello spazio che il datapath sa davvero esprimere, e il vettore Python si costruisce allo stesso modo. Confrontare due implementazioni su ingressi che una delle due non potrebbe mai ricevere non direbbe niente su nessuna delle due.

5.5 Il risultato

Trecento ingressi per scenario, seme fisso, pipeline P1.

Scenario	float vs int8	int8 Python vs eBPF	decisioni
ones	100,00 %	100,00 %	300/300
small	98,00 %	100,00 %	300/300
mixed	100,00 %	100,00 %	300/300
ipa_like	97,67 %	100,00 %	300/300
sparse	99,33 %	100,00 %	300/300
deep	98,67 %	100,00 %	300/300
large	99,67 %	100,00 %	300/300

Sette scenari su sette, equivalenza numerica esatta. La colonna di sinistra — fra 97,67 % e 100 % — è la quantizzazione, cioè una proprietà del modello e della scala scelta, non un difetto.

5.6 Ma ci è arrivato trovando due difetti

Il valore del test non sta nella tabella: sta nel fatto che la prima esecuzione **non** dava 7/7. Dava 1/7, e le due cause erano di natura diversa.

Il primo: il banco

Cinque scenari su sette stavano fra il 71 % e l'85 %. Il banco non comunicava al kernel la porta d'ingresso: `prog_test_run` accetta un parametro `ingress_ifindex` ma lo **ignora** — gira senza `ctx_in`, e il programma vede sempre l'`ifindex` del dispositivo di prova del kernel. Il riferimento Python usava una porta variabile, il datapath ne vedeva sempre nessuna, e le due metà costruivano due ingressi diversi.

Il segnale diagnostico è stato più utile del sintomo: dopo la prima correzione le percentuali non si erano mosse **di un centesimo**. Codice che non cambia niente non è codice sbagliato: è codice mai eseguito. Confermato senza kernel, simulando “il datapath vede porta 0”: le percentuali osservate si riproducono con scarto **0,00** su sei scenari.

Il secondo: la scala compilata

Resisteva `small`, al 70,33 %. Con una coincidenza che era la risposta: le righe *int8 Python vs riferimento* e *int8 Python vs eBPF* davano lo stesso identico numero, e *riferimento vs eBPF* dava 100 %. Il datapath non sbagliava un conto: eseguiva **esattamente** una configurazione sbagliata.

Tutte e tre le pipeline fissavano la scala della feature `ttl` a **30** in fase di compilazione — `feature_scale()` in P1, `T2_TTL_SCALE` in P2, `ML_TTL_SCALE` in P3 — e la struttura del descrittore letto a runtime, `struct feat_ent`, non aveva un campo dove metterla. Un modello addestrato su `ttl/16` veniva eseguito con `ttl/30`. Gli scenari `small` (16), `large` (64) e `ones` (8) sono esattamente quei modelli.

Sul checkpoint depositato, che usa 30, il difetto è invisibile.

La genericità di P2 e P3 — “il descrittore si legge a runtime, quindi qualunque modello va bene” — era vera per la forma, il numero di feature e gli offset, e **falsa per la scala**: l'unico numero del descrittore che deve seguire il modello era l'unico compilato dentro. Un'affermazione di genericità va verificata voce per voce, non in blocco.

5.7 La correzione

La scala ora viaggia nel descrittore. Il campo c'era già: `struct feat_ent` aveva un byte di riempimento inutilizzato, e le scale in gioco (8, 16, 30, 64) ci stanno.

Dove	Che cosa cambia
<code>model_meta</code>	<code>feature_scale_of(feat)</code> risponde per <i>feature</i> e non per <i>tipo</i> ; la scala entra nel descrittore risolto.
P1	compila la scala del descrittore invece di quella del catalogo.
P2 / P3	<code>_pad</code> diventa <code>scale</code> ; la divisione la legge dalla mappa. Con 0 si ricade sul default compilato, quindi i descrittori scritti prima restano validi.

Il modello depositato risolve a 30 come prima: nessuna regressione.

Un effetto collaterale che il compilatore ha fermato

Passare da una costante a una variabile ha rotto la compilazione di P2: “*unsupported signed division*”. Finché il divisore era un `#define` clang lo trasformava in uno spostamento di bit; da variabile diventa una divisione vera, e **BPF non ha la divisione con segno**. Il numeratore qui ce l’ha, perché i pesi possono essere negativi.

La correzione divide il valore assoluto senza segno e rimette il segno, con troncamento verso lo zero — lo stesso arrotondamento del riferimento Python, perché sui negativi le due implementazioni coincidono solo se arrotondano allo stesso modo.

5.8 Che cosa è costata, e la prova che regge

Una correzione che tocca il codice che gira per ogni pacchetto va pagata e misurata, non solo dichiarata. La suite kernel, rieseguita sul modello depositato:

Pipeline	istruzioni prima	dopo	differenza	
baseline	155	155	invariata	
P1 hardcoded	1 026	1 026	invariata	
P2 template	14 628	14 985	+357	(+2,4 %)
P3 modular	12 031	12 349	+318	(+2,6 %)

Il costo è **due virgola cinque per cento** sulle due pipeline generiche: il codice della divisione a divisore variabile più la lettura della scala dalla mappa. Che baseline e P1 restino *identiche al conteggio* è la verifica che il cambiamento non ha toccato percorsi che non doveva toccare — P1 compila la scala come prima, solo presa da un posto diverso.

L’esito funzionale è kernel suite: PASS — le quattro pipeline, la gestione del TTL, il reroute su link caduto, le architetture alternative e i due modelli concorrenti registrati insieme su P2 e su P3. E nel registro del descrittore si legge la scala arrivare a destinazione:

```
model_desc[0] = feats=[
(1,6,0,1) (2,6,6,1) (3,1,12,30) (4,52,13,1)
```

Il quarto campo è la scala: `t1` porta 30 fin dentro la mappa, e non è più un numero scritto nel sorgente.

5.9 Tre difetti, e chi li ha trovati

Vale la pena tenere il conto, perché dice qualcosa sul metodo più che sul codice.

Difetto	Di chi	Trovato da
Porta d’ingresso non detta al kernel	banco	il confronto, al primo run
Scala del TTL compilata a 30	datapath	il confronto, tolto il primo
Divisione con segno, non ammessa da BPF	correzione	il compilatore
Funzione nel ramo sbagliato di <code>#ifndef</code>	correzione	il compilatore, al secondo giro

Gli ultimi due sono stati introdotti *riparando* il secondo, e nessuno dei due sarebbe stato visibile senza compilare davvero. Da lì è nato `check_source_visibility.py`: i sorgenti eBPF di questo progetto non sono file ma stringhe, concatenate in combinazioni diverse dai chiamanti e ritagliate dal preprocessore con macro diverse. Una funzione può stare prima

del suo uso *nel file* e sparire lo stesso nella combinazione che conta. Il controllo valuta le guardie e verifica la visibilità in tutte e cinque le combinazioni realmente compilate — senza root e senza BCC, quindi prima di spedire e non dopo.

5.10 Che cosa questo capitolo dimostra

Che le tre pipeline calcolano ciò che il modello dice, su sette modelli che nessuno ha scelto per farle funzionare. E, di passaggio, che l'affermazione era falsa prima di essere verificata — il che è il motivo per cui la verifica serviva.

Dalla chiamata di sistema al kernel vero

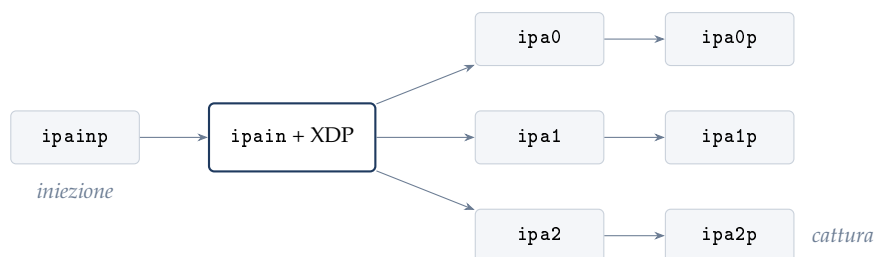
6.1 Che cosa provavano i test di prima

Fino a settembre ogni misura veniva da una chiamata di sistema, `BPF_PROG_TEST_RUN`: si dà al programma un buffer, lo si esegue N volte, e il kernel riporta quanti nanosecondi in media.

È utile e onesto per confrontare le tre pipeline fra loro. Ma non tocca un'interfaccia, non ha un driver, non consegna niente. Il programma poteva dire *"redirigerei sulla porta 3"* e nessuno verificava che un pacchetto ne uscisse davvero.

6.2 Il banco nuovo

Il banco è fatto di coppie di interfacce virtuali (veth) create sull'host: niente di più, e abbastanza vicino al vero da far girare XDP nella modalità in cui gira su un nodo.



Si inietta un frame da un capo, il programma XDP gira davvero, e si legge da quale interfaccia è uscito. Questo risponde alla domanda che `BPF_PROG_TEST_RUN` non può porre.

Due dettagli che sembrano minori e non lo sono

Modalità di attacco. XDP si può agganciare in due modi: *native*, dentro il driver, prima che esista la `sk_buff`; oppure *generic*, dopo. Il default è *native*, la modalità *davvero in effetto* viene riletta dal kernel dopo l'aggancio, e un fallimento è un errore, non una retrocessione silenziosa: una misura fatta in *generic* senza saperlo non si potrebbe attribuire a niente.

Il vincolo delle veth. Redirigere *verso* una veth funziona solo se il lato ricevente ha a sua volta un programma XDP agganciato: il kernel alloca le code necessarie solo in quel caso. Senza, il redirect fallisce in silenzio e il pacchetto sparisce — indistinguibile da un modello che decide di scartare tutto. Il banco aggancia quindi un programma minimo a ogni capo.

6.3 Tre difetti del test, trovati misurando

Il banco nuovo ha trovato più difetti nei *test* che nel prodotto. Vale la pena elencarli, perché il motivo per cui erano invisibili è sempre lo stesso.

1. Il TTL non muove questo modello

La prima versione del test variava il TTL da 2 a 8 e riportava *"7 verifiche su 7 superate"*. Misurando lo spazio raggiungibile si scopre che con tutti i link attivi il modello risponde **sempre classe 2**, per qualsiasi TTL. Quel 7 su 7 era **un caso solo, ripetuto sette volte**.

È `link_state` la feature che sposta la decisione. Il test adesso *cerca* un ingresso per ogni classe, invece di spazzare alla cieca.

2. Un pacchetto del kernel contato come risultato

Su una veth appena creata il kernel manda del traffico suo (IPv6 multicast). Una volta è stato catturato e contato come la risposta della pipeline. È passato per caso, perché era sulla porta giusta.

Adesso la cattura filtra sui campi che il datapath non riscrive, e IPv6 viene disabilitato sulle interfacce del banco.

3. La porta 0 era invisibile per costruzione

L'ingresso era anche la porta logica 0. Una classe che inoltrava su quella porta rimandava il frame fuori dall'interfaccia da cui era entrato — dove la cattura non lo distingue da quello appena iniettato. Una decisione *corretta* sulla porta 0 non era osservabile.

Il banco ha adesso un ingresso dedicato, separato da tutte le porte.

Un test che non può fallire non è un test. In tutti e tre i casi il risultato era verde e privo di contenuto, e in tutti e tre il difetto è emerso solo mettendo in mezzo un kernel vero.

6.4 Il risultato

Tutte e tre le pipeline superano oggi la verifica end-to-end su un datapath reale: XDP native, redirect vero, pacchetto ripreso sulla porta d'uscita, e scarto verificato dal *contatore* e non dal silenzio — perché "non è arrivato niente" è anche ciò che si osserva quando il redirect si perde.

La feature che non c'era

7.1 Il sintomo

Il modello ha quattro ingressi. Uno di questi è *da quale interfaccia è arrivato il pacchetto*, codificato *one-hot*: un vettore di sei posizioni con un solo uno.

Il datapath leggeva l'indice dell'interfaccia dato dal kernel — l' *ifindex* — e lo usava direttamente come posizione nel vettore, con un controllo di validità:

$$1 \leq \text{ifindex} \leq n_{\text{interfacce}}.$$

Gli *ifindex* però li assegna il kernel, in ordine di creazione. Su un sistema appena avviato valgono 2, 3, 4. Su una macchina che ha creato qualche interfaccia virtuale valgono 205, 217, 229.

Il controllo falliva sempre. **Nessun bit veniva acceso.** Il modello girava di fatto su tre ingressi invece di quattro.



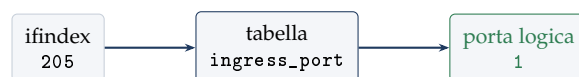
7.2 Perché nessun test lo vedeva

Perché nessun test controllava che quella feature *contribuisse* qualcosa. Riferimento e datapath la ignoravano entrambi, quindi **concordavano** — e l'accordo veniva registrato come successo.

Una pipeline aveva in effetti una tabella di traduzione, ma compilata dentro il programma come [2, 3, 4, 5, 6, 7]: la stessa assunzione, scritta in un altro modo. Le altre due usavano l'indice grezzo. Le tre pipeline sceglievano quindi *colonne diverse* per lo stesso pacchetto — una discrepanza documentata in un commento e mai risolta.

7.3 La correzione

La traduzione è diventata una **tabella a runtime**, riempita dal piano di controllo leggendo le interfacce del nodo:



È il **simmetrico** della tabella già usata in uscita: là "quale interfaccia realizza la porta che il modello ha scelto", qui "quale porta è l'interfaccia da cui il pacchetto è arrivato". Entrambe sono fatti del nodo, entrambe si risolvono quando il nodo esiste.

7.4 La prova che adesso è viva

Prima e dopo la correzione, gli ingressi che raggiungono ciascuna classe **cambiano**:

classe	prima (feature morta)	dopo (feature viva)
0	100110, ttl 27	100111, ttl 20
1	010000, ttl 2	010000, ttl 14
5	000000, ttl 2	000000, ttl 12

Stesso stato dei link, TTL diverso per ottenere la stessa classe: l'ingresso in più contribuisce. E il TTL ora sposta la decisione, cosa che prima non faceva.

7.5 Il prezzo, misurato

Una lettura di tabella in più per pacchetto:

	prima	dopo
letture per pacchetto (P1)	4,0	5,0
letture per pacchetto (P3)	26,0	28,0
istruzioni (P1)	997	971

Le istruzioni di P1 *calano*: la tabella ha sostituito un blocco switch a sei casi, e una lettura costa meno di un salto.

Una lettura in più arriverà poco dopo, per la stessa ragione applicata a un altro ingresso: è il capitolo che segue. Le cifre finali sono 6,0 letture per P1 e 29,0 per P3.

Il nodo non sapeva di essere un nodo

8.1 L'ingresso più grande di tutti

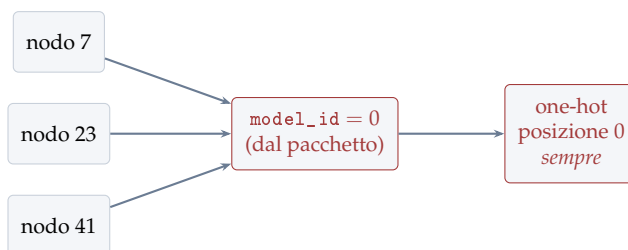
Dei quattro ingressi del modello, quello che pesa di più è *quale nodo sta elaborando il pacchetto*. Su Germany50 sono 52 nodi, quindi un vettore one-hot di 52 posizioni: **208 dei 319 pesi** della rete, quasi due terzi.

8.2 Il sintomo

Il datapath riempiva quel vettore con il campo `model_id` preso dall'intestazione IPA del pacchetto.

Sono due cose diverse. `model_id` dice *quale modello* il pacchetto chiede di usare; il vettore vuole sapere *quale nodo* lo sta elaborando. E il `model_id` non cambia lungo il percorso: è scritto dal mittente e viaggia identico fino alla destinazione.

Con un solo modello registrato — il caso normale — **ogni nodo della rete accendeva la stessa identica posizione**. Due terzi dei pesi della rete non distinguevano nulla.



8.3 La correzione, e perché una tabella e non un array

Stessa forma della correzione precedente: una **tabella letta a runtime**, riempita quando il nodo viene messo in servizio, dalla variabile d'ambiente `IPA_NODE_ID` o dall'indice del nodo nella topologia.

Un dettaglio che sembra tecnico e non lo è: la tabella è di tipo *hash* e non *array*. Un array in questo ambiente è preallocato e riempito di zeri, quindi una lettura riesce **sempre**: "non configurato" e "sono il nodo 0" tornerebbero lo stesso valore, indistinguibili. Sarebbe esattamente il difetto che la tabella serve a togliere. Con un hash, finché nessuno scrive non c'è voce: il vettore resta spento, ed è la risposta onesta.

Questo errore è stato commesso e corretto: la prima versione usava un array, e un test su un'architettura alternativa è fallito 5 volte su 5 prima che se ne capisse il motivo.

8.4 Una pipeline ha resistito un giorno

P1 e P2 hanno accettato la modifica subito. P3 no: il verificatore del kernel ha **rifiutato il programma**, dodici configurazioni di fila.

Il motivo è controintuitivo e vale la pena averlo scritto. Il verificatore non misura quanto è lungo il programma: esplora i *cammini* possibili, e il numero di stati che deve tenere è il **prodotto** dei valori indipendenti che segue dentro un ciclo srotolato, non la loro somma.

$$\text{stati} \sim v_1 \times v_2 \times \dots \times v_k$$

L'unica versione che caricava era quella in cui l'indice del nodo *non era un valore indipendente*: `nodo = model_id`, cioè lo stesso registro che il verificatore stava già seguendo. Un valore, non due.

Escluse una a una, ognuna con la sua prova: il tipo di tabella, la sua dichiarazione, il valore, il tipo del dato, il numero di letture, la larghezza in bit. Nessuna era la causa.

Quello che ha liberato spazio non toccava il nodo: un **tetto** di tutt'altra feature. Il programma teneva in vita otto posizioni per un vettore di occupazione delle code che il modello depositato *non dichiara nemmeno*. Portato quel tetto da 8 a 1, il programma ha caricato — e più piccolo di prima.

Questi limiti sono **scogliere, non pendenze**. Nello stesso giro, ridurre *anche* un secondo tetto — cioè chiedere *meno* — ha fatto fallire di nuovo il caricamento. Un tetto qui si cambia e **si rimisura**; non si ragiona.

8.5 La prova che il nodo è vivo

Stesso programma compilato, due esecuzioni che differiscono solo per la tabella del nodo:

esecuzione	esito sul pacchetto
senza indice di nodo	scartato
indice di nodo = 7	non scartato

Nient'altro è cambiato fra le due. L'ingresso sposta la decisione.

Il verificatore non conta le istruzioni

9.1 Due monete diverse

Il tetto sulle code portato a 1 era una perdita vera: a 1, un modello che dichiara davvero quella feature viene *rifiutato*. Per recuperarlo serviva margine di verifica, e il margine è arrivato da una riscrittura che non toglie niente.

9.2 Il ciclo era girato dalla parte sbagliata

Il primo strato della rete si calcola così: per ogni neurone, si sommano i contributi di ogni feature dichiarata nel descrittore.

Il codice lo faceva nell'ordine ovvio — neuroni fuori, feature dentro:

```
per ogni neurone j (8 volte)
  per ogni feature f (4 volte)
    di che tipo e' f?      <- catena a 5 rami
    di che larghezza e' f? <- controllo per ogni posizione
  accumula
```

Ma *di che tipo* e *di che larghezza* è una feature non dipende dal neurone. Quella catena a cinque rami veniva quindi rivalutata $8 \times 4 = 32$ volte per pacchetto, e il controllo sulla larghezza di un vettore denso $8 \times 8 = 64$ volte, per ottenere ogni volta la stessa risposta.

Girato al contrario — feature fuori, neuroni dentro — diventa:

```
per ogni feature f (4 volte)
  di che tipo e' f?      <- 4 volte, non 32
  di che larghezza e' f? <- 8 volte, non 64
  per ogni neurone j (8 volte)
    accumula              <- nessun ramo qui dentro
```

9.3 Perché il risultato non cambia di un bit

Cambia solo l'**ordine in cui i termini vengono sommati**. I termini sono gli stessi, calcolati allo stesso modo. E la somma fra interi a 64 bit in complemento a due è associativa e commutativa — anche in caso di traboccamento, che si comporta identicamente nei due ordini:

$$(a + b) + c = a + (b + c).$$

Ogni accumulatore finisce quindi **bit per bit identico**. Non "circa uguale": identico.

9.4 Il risultato, che sembra un errore

	prima	dopo
P2, programma principale	16 144	14 587
P3, primo strato	6 417	6 850
P3, tetto sulle code caricabile	1	8

P3 è **cresciuto** di 433 istruzioni ed è diventato molto più economico da verificare. Prima della riscrittura, col tetto delle code a 8 non caricava niente; dopo, carica a 10 207 istruzioni.

Dimensione del programma e complessità di verifica sono **monete diverse**. Il verificatore paga i cammini.

9.5 Un effetto collaterale che vale una riga di tabella

Contare quante letture di tabella fa un programma per pacchetto richiede di ricompilarlo con un contatore su ogni punto di lettura. Le versioni strumentate di P2 e P3 **sforavano il verificatore**: per questo la riga "letture di tabella" riportava *n.d.* per entrambe.

Con il margine liberato dalla riscrittura, tutte e due caricano. La misura che era impossibile è diventata una riga di tabella — 11 letture per P2, 29 per P3 — senza che nessuno abbia riscritto il contatore.

Un bivio per ogni neurone (28 settembre)

10.1 Il sintomo

Sulla macchina di laboratorio due cose smettevano di caricarsi:

- **P2 con tre o più strati nascosti:** rifiutata dal verificatore;
- **P1 oltre qualche centinaio di pesi:** rifiutata anche lei, e prima la forma larga.

In entrambi i casi il messaggio era lo stesso:

```
BPF program is too large. Processed 1000001 insns
```

Nella macchina virtuale P2 si caricava fino a sei strati. Il sorgente era lo stesso, quindi la prima spiegazione è stata "cambia il kernel". Era sbagliata.

10.2 Che cosa fa il verificatore

Prima di accettare un programma eBPF, il kernel lo fa controllare dal **verificatore**. Il verificatore non esegue il programma su un pacchetto vero: lo percorre *su tutte le strade possibili*, per essere sicuro che nessuna faccia qualcosa di proibito. Ogni `if` è un bivio, e il verificatore prende tutti e due i rami. Ha però un budget: **un milione di passi**. Finito il budget, rifiuta il programma, per quanto corto sia.

Il programma P2 a tre strati ha circa 16 000 istruzioni: il verificatore ne percorreva un milione. Stava rileggendo gli stessi pezzi di codice molte volte, una per ogni strada.

10.3 Dove stavano i bivi: la ReLU

Ogni neurone nascosto applica la ReLU: *se il valore è positivo tienilo, altrimenti metti zero*. Nel codice era scritta esattamente così:

```
h = a > 0 ? a : 0; // un if: due strade
```

Un bivio per neurone. Con tre neuroni in fila le strade sono

$$2 \times 2 \times 2 = 8,$$

una per ogni combinazione di "positivo / negativo":

neurone 1	neurone 2	neurone 3	
+	+	+	strada 1
+	+	−	strada 2
+	−	+	strada 3
	...		...
−	−	−	strada 8

Con i 16 neuroni del modello largo di P1 sono $2^{16} = 65\,536$ strade. Il verificatore ha un meccanismo per riconoscere due strade "equivalenti" e percorrerne una sola (la *potatura*), ma qui riusciva a riunirne poche: i numeri qui sotto dicono che la gran parte delle strade veniva percorsa per intero.

Il verificatore scrive quanti passi ha fatto, se glielo si chiede (`diag_verifier.py`, statistiche con `log_level = 4`). Il P1 largo, 3 486 istruzioni, arrivava al milione: **287 volte** la sua lunghezza. P2 a *uno* strato era già a 799 563 passi, l'80% del budget. Il margine era così piccolo che una differenza qualsiasi fra due kernel bastava a superarlo: per questo nella macchina virtuale passava e qui no.

10.4 La correzione: la ReLU senza bivio

La ReLU si può calcolare con sola aritmetica, senza if:

```
m = a >> 63;           // tutti 1 se a e' negativo, tutti 0 altrimenti
h = a & ~m;             // a se positivo, 0 se negativo
```

Lo spostamento a destra di 63 posizioni di un intero con segno a 64 bit "ricopia" il bit del segno su tutti i bit. Due esempi, mostrati su 8 bit per leggibilità (a 64 bit è identico):

	$a = 5$	$a = -3$
a in bit	00000101	11111101
$m = a \gg 63$	00000000	11111111
$\sim m$	11111111	00000000
$a \& \sim m$	00000101 = 5	00000000 = 0

Con a positivo la maschera lascia passare tutto e resta a ; con a negativo la maschera azzerava tutto. È la ReLU, **bit per bit la stessa**, ma senza bivio: il verificatore ha una strada sola da percorrere.

Il dettaglio che la fa funzionare

Scritta così e basta, non cambia niente: clang riconosce che $a \& \sim(a \gg 63)$ è "il massimo fra a e 0" e, per ottimizzare, **rimette il bivio**. Nel bytecode compilato i salti tornano tutti (136 contro 120 nel P1 largo). Serve una riga che impedisca a clang di vedere dentro la maschera:

```
static __always_inline long long ipa_relu(long long a) {
    long long m = a >> 63;
    asm volatile("" : "+r"(m)); // barriera: nessuna istruzione, m invariato
    return a & ~m;
}
```

L'asm è vuoto: non produce nessuna istruzione e non cambia m . Dice solo a clang "di m non sai niente", e clang non può più riconoscere il massimo. La funzione è la stessa in P1 (`gen_full_c.py`), in P2 (il leaf di `ebpf_template_arch.py`) e in P3 (`ebpf_modular.py`). P3 caricava anche prima, perché ripete un solo strato generico: la ReLU è cambiata anche lì perché le tre pipeline calcolino l'attivazione con le stesse istruzioni, e il confronto fra loro resti alla pari.

10.5 Il risultato

Passi del verificatore, su un budget di 1 000 000:

	ReLU con bivio	ReLU senza bivio
P2, 1 strato	799 563	120 452
P2, 2 strati	417 292	120 557
P2, 3 strati	<i>rifiutata</i>	127 906
P2, 6 strati	<i>rifiutata</i>	136 642
P2, 20 strati	<i>rifiutata</i>	201 528
P1 larga, 1×16 ($\sim 1\,200$ pesi)	<i>rifiutata</i>	5 698
P1 profonda, 8×9 ($\sim 1\,300$ pesi)	229 039	8 516
P3, layer_first	27 183	26 755
P3, layer_hidden	186 489	27 979

Adesso P2 carica fino a venti strati, e a sei ne usa il 14% del budget. P1 carica il tier da $\sim 1\,200$ pesi in tutte le forme. P3 caricava già, ma lo strato che ripete (layer_hidden) costava al verificatore 117 volte la sua lunghezza: ora 17. layer_first quasi non cambia, perché la sua ReLU è una parte piccola di un programma dominato dalla lettura degli ingressi.

Il nuovo limite di P2 non è il verificatore

Oltre venti strati è **clang** a fermarsi, prima ancora del kernel:

```
LLVM ERROR: Branch target out of insn range
```

Un salto eBPF può arrivare al massimo a 32 767 istruzioni di distanza. I controlli all'inizio del programma (*se il pacchetto non va bene, esci*) saltano fino in fondo, e ogni strato allunga il programma di circa mille istruzioni: a venti strati il programma ne ha 32 388, appena sotto. Il terzo limite è la tabella dei pesi, 1 024 posti: 37 strati al massimo a larghezza 4, 7 a larghezza 8.

10.6 Che cosa costa

Il calcolo della rete è identico: i test che confrontano il C di P1 con il riferimento Python danno gli stessi logit su ogni modello sintetico (test_synth.py), e p1_c_eval.py ora valuta anche ipa_relu leggendone il corpo. Cambiano invece le istruzioni: circa tre per neurone al posto di due, in cambio di nessun salto. Rimisurato tutto il 28 settembre:

	baseline	P1	P2	P3
programma da solo, 27 settembre	22 ns	47 ns	199 ns	323 ns
programma da solo, 28 settembre	18 ns	52 ns	195 ns	315 ns
traffico vero, sopra la baseline, 27	—	38 ns	224 ns	368 ns
traffico vero, sopra la baseline, 28	—	38 ns	226 ns	364 ns

La baseline, che di ReLU non ne ha, è scesa di 4 ns fra i due giorni, e P2 e P3 con lei: la macchina era un po' più veloce. P1 invece è **salita** di 5 ns, quindi rispetto alla baseline la ReLU senza bivio le costa circa 9 ns nel banco a pacchetto ripetuto. Lì lo stesso pacchetto passa milioni di volte, il processore indovina sempre da che parte va ogni bivio, e un bivio indovinato è quasi gratis: toglierlo e aggiungere tre istruzioni sempre eseguite si vede. In P2 e P3 la ReLU è una parte minima di un programma molto più grande, e non si vede.

Sul traffico vero la differenza sparisce anche per P1: 38 ns sopra la baseline prima e dopo. Il prezzo è quindi piccolo e si paga soprattutto nel banco; in cambio P1 carica modelli fino a quattro volte più grandi, P2 fino a venti strati, e le tre pipeline calcolano l'attivazione con le stesse istruzioni.

"Cambia il kernel" era una spiegazione comoda e sbagliata. Il programma lavorava già all'80% del budget di verifica, e il colpevole era una riga: un `if` per neurone. Come nel capitolo precedente, il verificatore non paga le istruzioni ma le **strade**: togliere un bivio vale più che togliere cento istruzioni.

I numeri, e che cosa dicono davvero

11.1 La tabella

Modello 65-4-4-7, scala 24, `test_suite --only kernel`, P1 come oggetto AOT (quello che va sui nodi). Misurato il 28 settembre sulla macchina di laboratorio, con la ReLU senza bivio (capitolo 10), il core del programma fisso a 3 494 MHz e isolato dal resto (capitolo 15).

	baseline	P1	P2	P3
istruzioni caricate	155	1 064	15 142	12 394
salti fra programmi	0	1	1	3
letture di tabella	3	6	11	29
memoria delle tabelle	280 B	2 516 B	141 744 B	171 572 B
latenza (minimo di 7)	18 ns	52 ns	195 ns	315 ns

Il *baseline* è un programma che analizza il pacchetto e lo rimanda fuori senza alcuna inferenza: è il pavimento contro cui si legge tutto il resto.

Tutte le colonne vengono dalla **stessa esecuzione**, sulla stessa macchina, nello stesso minuto.

11.2 Primo risultato: la dimensione non predice la velocità

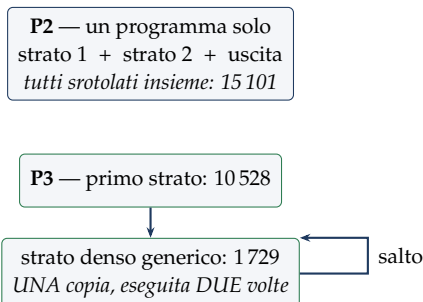
P3 ha il 18% di istruzioni in meno di P2 ed è il 62% più lento.

Il motivo sta nelle due righe centrali: tre salti fra programmi invece di uno, e ventinove letture di tabella per pacchetto invece di undici.

Il conteggio delle istruzioni misura quanto è *grande* il programma caricato, non quanto *lavoro* fa per pacchetto. In questo regime a dominare sono le letture di memoria e i salti, non l'aritmetica.

Perché P2 è più grande, pur essendo più veloce

Le due cose hanno la stessa causa: **P3 non tiene la rete intera in un programma.**



P3 srotola **un** solo strato denso generico e ci rientra, con un salto fra programmi, a ogni passo della rete. Così la *profondità* del modello non costa dimensione: un modello a sei strati gira sullo stesso identico binario. P2 deve srotolarli tutti, perché è un programma solo.

Quel riuso è esattamente ciò che P3 paga in latenza: ogni rientro è un salto più le letture che servono a ricostruire il contesto perduto. Il salto in sé costa quasi niente — misurato da solo, **un nanosecondo** — quindi quello che pesa sono le letture.

La stessa scelta di progetto spiega *entrambi* i numeri: P3 è più piccolo **perché** riusa un pezzo, ed è più lento **perché** riusarlo costa delle letture ogni volta.

11.3 Secondo risultato: quanto valgono le latenze assolute

Ogni cifra di latenza in tabella è il **minimo di sette misure** indipendenti. Guardando come sono distribuite quelle sette, dentro la singola esecuzione:

	baseline	P1	P2	P3
minimo	18 ns	52 ns	195 ns	315 ns
mediana	18 ns	53 ns	200 ns	320 ns
massimo	19 ns	53 ns	200 ns	322 ns
dispersione	6%	2%	3%	2%

La dispersione sta fra il 2 e il 6%: su 18 ns un nanosecondo è il 6%. È il risultato del condizionamento della macchina (capitolo 15): frequenza fissa e misurata, stati di sonno profondi spenti, il core del programma lasciato solo.

Le cifre assolute valgono **a quella frequenza**: il core del banco è fisso a 3,5 GHz. A un'altra frequenza cambiano quasi in proporzione; i **rapporti** fra le pipeline no.

Un caso in cui la regola è servita

Sull'asse dei nodi dell'analisi parametrica (capitolo seguente), P1.5 passa da 776 a 1718 istruzioni fra 10 e 100 nodi, e la latenza resta a 45–48 ns in tutti e cinque i punti. E il tetto sulle code di P3, portato da 1 a 8, aggiunge 3525 istruzioni al primo strato (da 7003 a 10528) senza cambiare niente di ciò che gira: il modello depositato non dichiara la feature delle code, quindi quel ramo non viene mai eseguito.

Una metrica che si sposta del 50% senza che cambi nulla di ciò che gira **non è una misura di costo**. Il numero di istruzioni va letto come "quanto è grande il binario", mai come "quanto lavora".

11.4 Terzo risultato: la genericità ha un costo di verifica

P2 serve qualsiasi modello che stia sotto certi tetti dichiarati a priori (numero massimo di neuroni, di ingressi, di feature). È il suo pregio: non ricompila mai.

Il conto però lo paga il verificatore del kernel, che percorre ogni cammino del corpo srotolato. Con i tetti larghi il kernel si arrende:

```
processed 1000001 insns (limit 1000000) ... total_states 13864
```

Da notare: il programma è lungo circa 14 900 istruzioni, **ampiamente dentro** il limite di dimensione. Quello che finisce è la *complessità di verifica*, che è un limite diverso e cresce col **prodotto** dei tetti:

$$\text{costo di verifica} \sim H_{\max} \times F_{\max} \times N_{\max}.$$

Lo stesso limite fermava P2 oltre due strati nascosti, e la P1 oltre qualche centinaio di pesi, finché la ReLU non ha perso il suo bivio (capitolo 10).

È la misura concreta di quanto un programma "generico" possa esserlo. La genericità di P2 non è gratis: si paga in stati del verificatore, e il tetto si tocca con numeri ragionevoli.

11.5 Quarto risultato: togliere il compilatore dal nodo

P1 scrive i pesi *dentro* il codice come numeri letterali. È ciò che la rende veloce — il compilatore può far sparire le moltiplicazioni per zero e trasformare in scorrimenti quelle per potenze di due — ma ogni volta che il modello cambia bisogna ricompilare. Con BCC la compilazione avviene sul nodo: **75 ms** per modello, contro 2 ms di caricamento nel kernel.

L'alternativa è compilare *prima*, su una macchina qualunque, e spedire al nodo l'oggetto già pronto. Al momento del deploy il nodo apre il file e lo carica, senza compilatore:

oggetto precompilato (AOT)	
compilazione, una volta, fuori dal nodo	75,8 ms
messa in servizio sul nodo (apri, verifica, JIT)	1,12 ms
istruzioni	1 064
latenza per pacchetto	51–52 ns

Il nodo non ha più bisogno di un compilatore, e il costo per pacchetto è lo stesso di tutte le altre misure: **ogni cifra di P1 in questo quaderno è dell'oggetto AOT**. La riduzione fatta dal compilatore sui pesi letterali resta dentro l'oggetto.

Vale solo quando i modelli sono noti in anticipo, che è l'ipotesi sotto cui P1 esiste.

11.6 Che cosa questi numeri non dicono

BPF_PROG_TEST_RUN esegue il programma in un ciclo sullo stesso buffer. Niente scheda di rete, niente driver, nessuna allocazione, nessuna pressione di cache da traffico vero.

Il valore in "milioni di pacchetti al secondo" è semplicemente $1/\text{latenza}$: un **picco teorico**, non un throughput retto. Per quello serve traffico vero, generato a ritmo, e la misura di quanti pacchetti si perdono (capitolo 14).

Come cresce il costo

12.1 La domanda

Fin qui tutto è misurato su *un* modello e *una* topologia. La domanda che manca è come cambia il costo quando la rete o il modello diventano più grandi — e soprattutto se le pipeline hanno **pendenze diverse**.

Se le pendenze fossero uguali, averne più di una sarebbe un esercizio. Non lo sono, ed è questo il capitolo che lo mostra.

Le pipeline confrontate sono **quattro**, e formano una scala di *quanto si sa quando il programma viene compilato*:

	compilato dentro	un binario per
P1 specializzata	pesi <i>e</i> indice di questo nodo	nodo
P1.5 hardcoded	pesi; il nodo si legge a runtime	modello
P2 template	solo i soffitti	tutto
P3 modular	anche la profondità è a runtime	tutto

La P1.5 è quella che i capitoli precedenti chiamano semplicemente P1: la *disposizione* delle feature è scritta nel codice, ma l'indice del nodo no. La P1 specializzata congela anche quello.

12.2 Come è impostata la misura

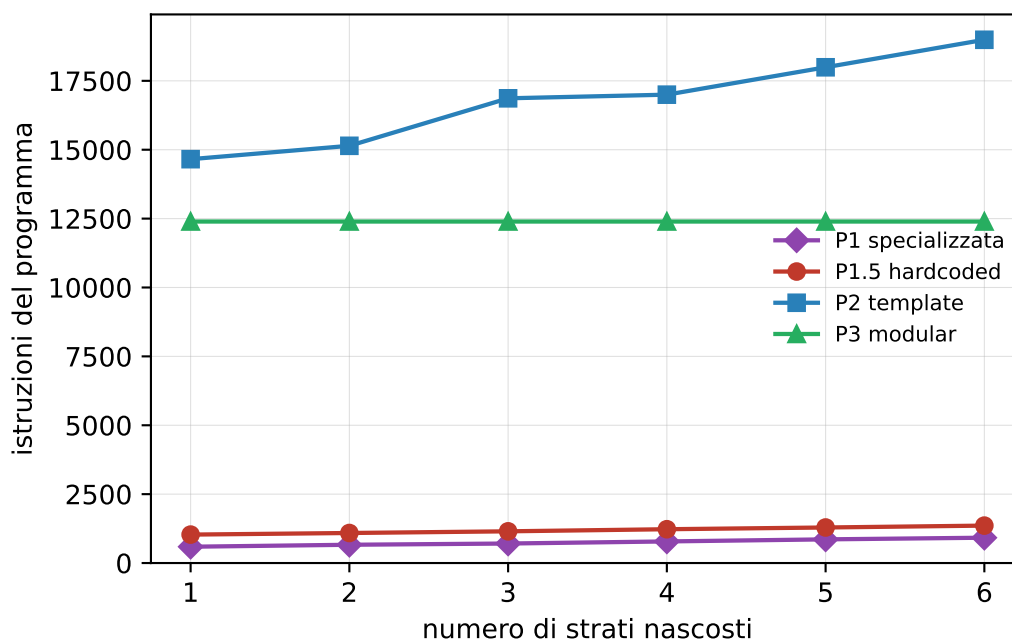
Sette assi, e su ciascuno si muove **una sola** variabile: tutto il resto resta fermo, così una curva si può attribuire a quello che c'è scritto sull'asse delle ascisse e a nient'altro.

asse	valori	si ferma perché
nodi della rete	10, 25, 52, 75, 100	il tetto sugli ingressi è 128
hidden layer	da 1 a 6	—
a parametri fermi	da 1 a 5 strati, 592 pesi	—
neuroni per layer	2, 4, 6, 8	il tetto compilato è 8
composizione dell'IV	4 varianti	0, 1 o 2 one-hot
pesi a zero	0% ... 90%	—
porte presenti	da 2 a 6	—

Tre accortezze che cambiano i risultati, non solo la forma:

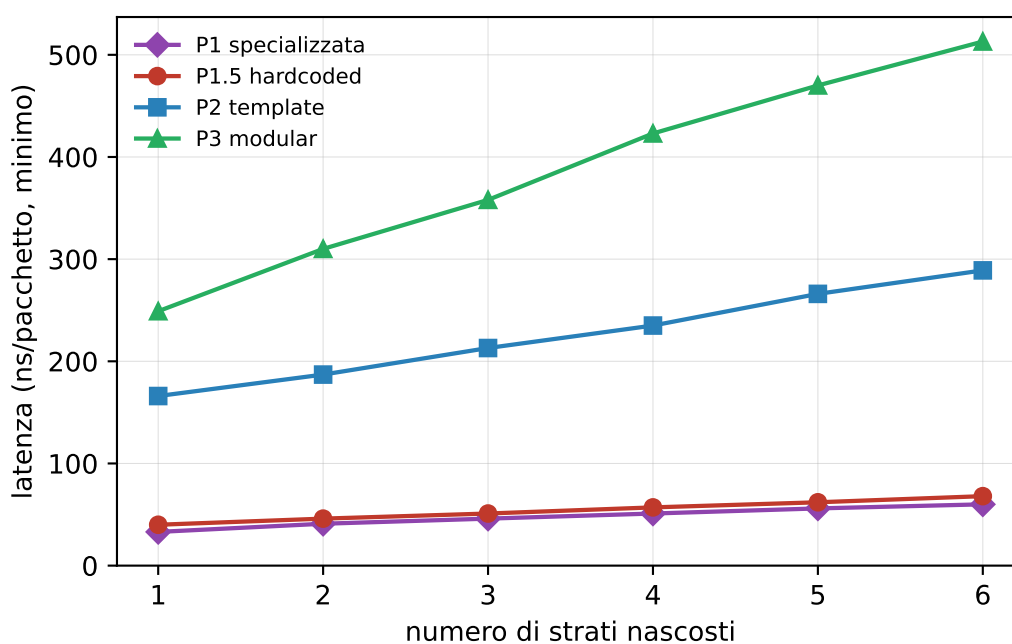
- **Ogni misura in un processo separato.** P1 può far abortire il compilatore in modo non recuperabile; e un rifiuto del verificatore è un *dato*, non un guasto dello strumento.
- **I pesi vengono da un sacchetto fisso**, e un modello più grande *estende* quello più piccolo invece di sostituirne tutti i pesi: in P1 il numero di istruzioni dipende dai valori dei pesi, e cambiarli tutti a ogni punto mescolerebbe il loro effetto con quello dell'asse.
- **Un allarme automatico sulle misure sporche:** se una pipeline ha lo stesso identico programma e la latenza varia più del doppio lungo l'asse, lo scarto è la macchina e lo strumento lo dice.

12.3 Profondità: chi paga in spazio e chi in tempo



P3 ha la stessa dimensione da uno a sei strati: 12 394 istruzioni in ogni punto. P2 cresce di circa 870 istruzioni per strato, da 14 657 a 18 990.

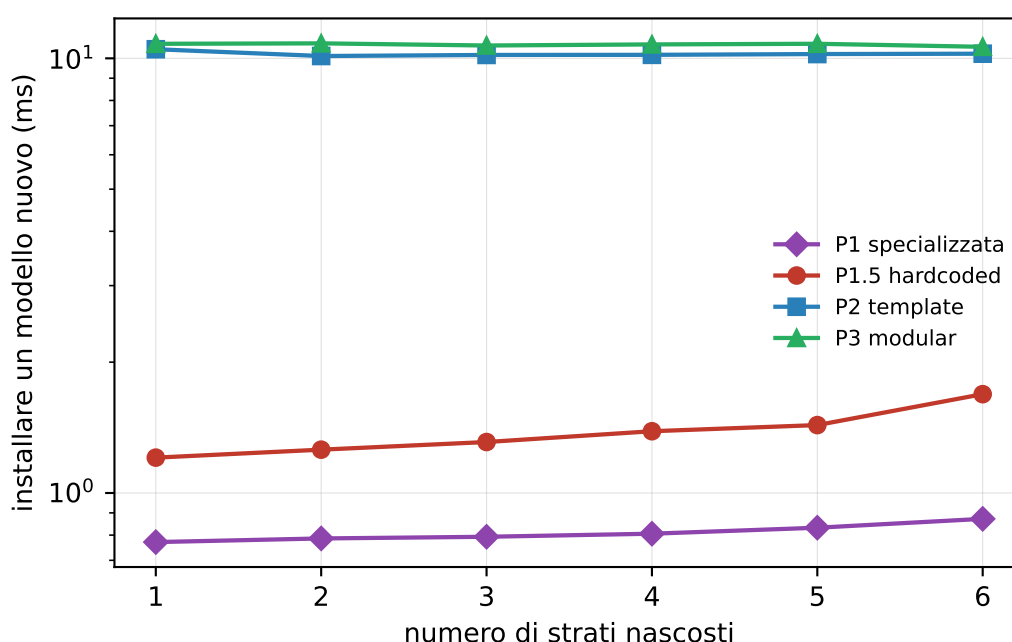
P3 srotola *un* solo strato denso generico e ci rientra a ogni passo della rete, quindi la profondità non entra nel programma. P2 deve srotolarli tutti, perché è un programma solo. Fino al 27 settembre P2 oltre due strati non si caricava: il verificatore si arrendeva, per colpa di un bivio in ogni ReLU (capitolo 10). Questa è la prima misura con tutti e sei i punti.



E P3 lo paga in tempo: circa 53 nanosecondi per strato (249 → 513 ns da uno a sei strati), a istruzioni identiche. **P2 ne paga circa 25** (166 → 289 ns): lo strato in più è codice in linea, niente salti e niente letture per ricostruire il contesto. Le due P1 crescono di circa 5 ns per strato (33 → 60 la specializzata, 40 → 68 la P1.5).

Le due figure hanno la *stessa* causa vista dai due lati. P3 è più piccolo **perché** riusa un pezzo, ed è più lento **perché** riusarlo costa un salto e delle letture ogni volta. È il compromesso della modularità, ridotto a due numeri. La differenza vera sulla profondità resta un'altra: P3 cambia profondità senza ricompilare, P2 deve ricompilare il programma.

12.4 Aggiornare il modello: con l'oggetto precompilato, pari



Con P1 costruita come oggetto precompilato, installare un modello nuovo su un nodo già in servizio costa **0,7–2,1 millisecondi** in P1 — il nodo apre e carica un file pronto — e **10–12 millisecondi** in P2 e P3, che scrivono in una tabella (pesi e semantica delle classi).

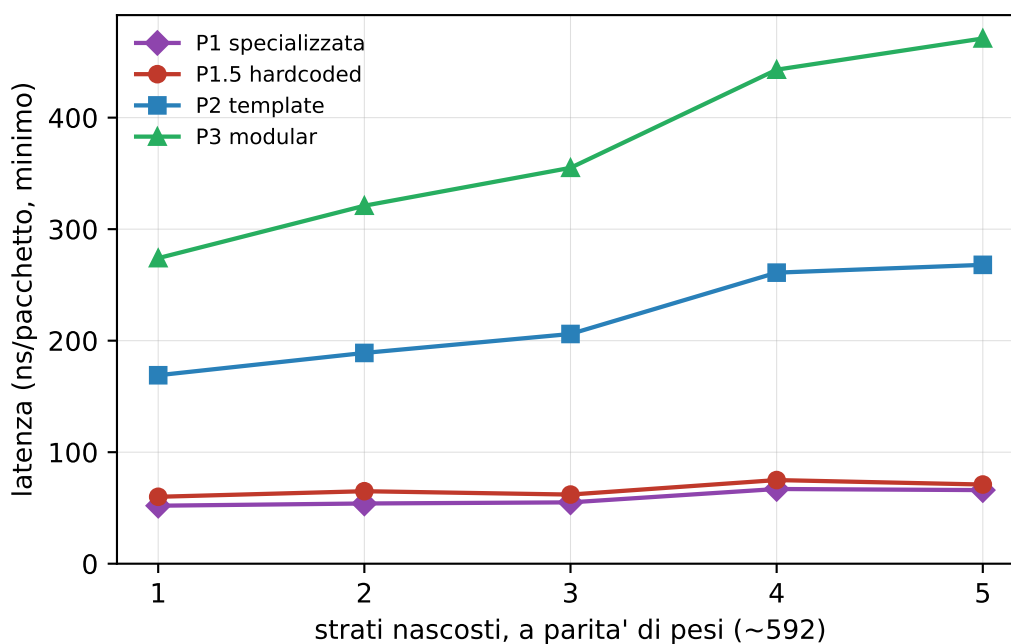
Contando anche il lavoro di P1 fuori dal nodo — generare il C, compilarlo con clang — si sta fra 66 e 120 ms. P2 e P3 compilano con BCC una volta sola, all'avvio del nodo (1,2–1,7 s), e mai più.

La differenza che resta fra le pipeline non è quindi il tempo di aggiornamento, ma *dove* serve il compilatore: P1 ne vuole uno per ogni modello nuovo, su qualche macchina; P2 e P3 no.

12.5 Larga o profonda? Dipende da chi la esegue

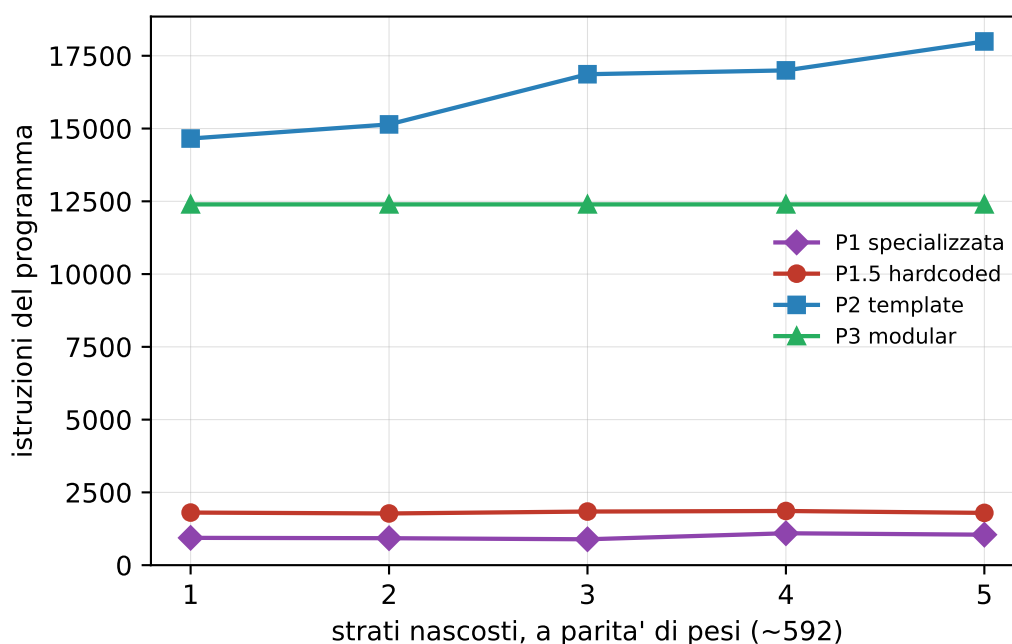
Dato un numero di parametri da spendere, conviene metterli in *un* layer largo o in *molti* stretti? È una domanda che si fa di solito parlando del modello, come se la risposta fosse una proprietà della rete neurale. Qui non lo è.

Cinque forme, tutte con lo stesso budget — 592 pesi, con uno scarto del 2% — disposto in modo diverso: da un solo strato da otto neuroni a cinque strati stretti.



A parità di parametri le due versioni hardcoded cambiano poco e non in modo monotono (52–67 ns la specializzata, 60–75 la P1.5: circa +25% fra gli estremi). P2 perde il 59% da uno a cinque strati (169 → 268 ns), P3 il 72% (274 → 471 ns).

La stessa scelta di architettura costa poco o costa molto, a seconda della pipeline su cui finisce.



P3 ha la riga **perfettamente orizzontale** — 12 394 istruzioni a tutti e cinque i punti — e perde comunque il 72% di velocità.

Dimensione costante e tempo crescente è la firma di un costo di transizione, non di calcolo. In P3 si vede allo stato puro: ciò che cresce non è il lavoro, è il numero di volte che il programma deve rientrare in sé stesso.

Con budget diversi, e ingressi diversi

Il banco dedicato (bench_depth_vs_width) prova tre budget e quattro vettori d'ingresso, sulla sola P1:

	larga	2 strati	4 strati	8 strati	
<i>~300 pesi</i>					
ingresso con due one-hot (65 valori)	39 ns	46	49	64	(+64%)
ingresso denso (11 valori)	93 ns	88	83	81	(-13%)
<i>~1 200 pesi</i>					
ingresso con due one-hot (65 valori)	110 ns	150	176	219	(+99%)
ingresso denso (11 valori)	stack	stack	320	291	(-9%)

P1 come oggetto AOT, ReLU senza bivio, 28 settembre. *stack*: clang non compila, la forma chiede più dei 512 byte di stack di eBPF.

Con un ingresso **grande e dominato da una one-hot** — il caso di IPA, 52 dei 65 valori — allargare batte approfondire, e a 1 200 pesi la rete a otto strati costa il doppio di quella larga. Con un ingresso **piccolo e denso** vince la rete profonda: più veloce del 13% a 300 pesi e anche più piccola (1 368 contro 1 535 istruzioni). Il motivo: una one-hot larga costa poco per pacchetto, perché il datapath ne legge *una colonna sola*; un ingresso denso invece fa moltiplicare le letture per la larghezza del primo strato.

I limiti di P1, prima e dopo il 28 settembre

Fino al 27 settembre, a $\sim 1\,200$ pesi sull'ingresso di IPA **caricava solo la forma a otto strati**: la larga, quella a due e quella a quattro strati venivano rifiutate dal verificatore (Processed 1000001 insn). La causa era un bivio in ogni ReLU (capitolo 10), e la forma larga ne ha di più in fila. Con la ReLU senza bivio carica tutto il tier.

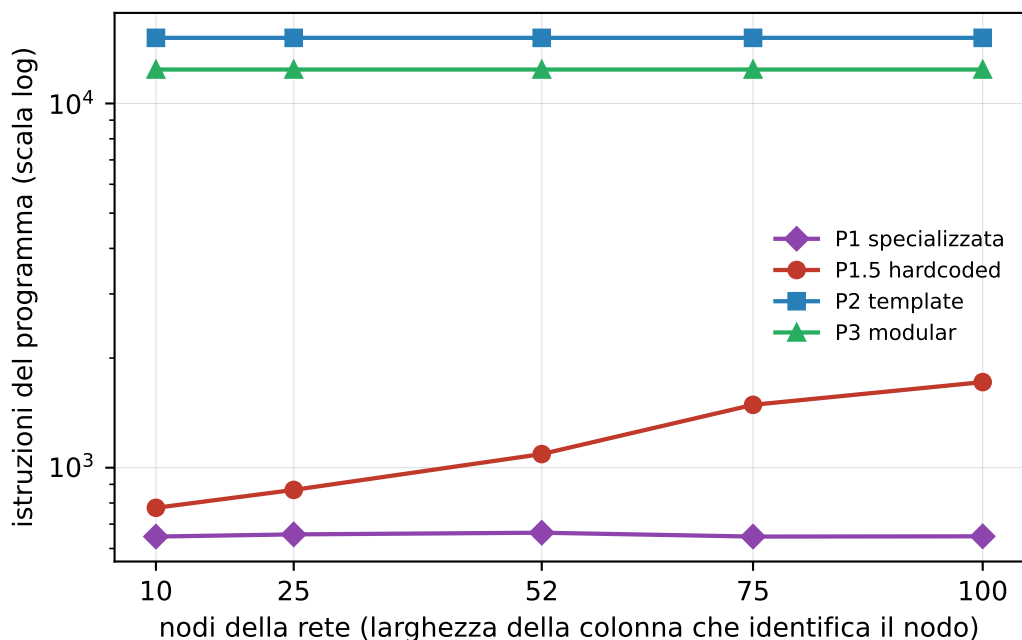
Il limite che resta è un altro, e non è del verificatore: lo **stack** di eBPF, 512 byte. Con l'ingresso denso a 1 200 pesi la forma larga (un solo strato da 63 neuroni) e quella a due strati da 26 chiedono più spazio per i valori intermedi di quanto il programma ne abbia, e clang si ferma prima di produrre l'oggetto. A $\sim 4\,700$ pesi succede a tutte le forme.

La frase giusta non è "le reti larghe sono preferibili a quelle profonde". È: *con un ingresso dominato da feature one-hot, allargare batte approfondire; con un ingresso piccolo e denso vince la rete profonda; e oltre qualche centinaio di pesi, in P1, è la forma larga a esaurire per prima lo stack.*

12.6 Congelare il nodo: conviene?

L'indice del nodo è una **costante del deployment**: un nodo non cambia di posto. Il codice però lo legge a runtime, e per farlo porta uno switch con un caso per ogni nodo della rete — 52 casi per Germany50, di cui ne esegue *uno*.

Congelandolo a tempo di generazione lo switch sparisce, e con lui la lettura della tabella. Restano quattro numeri costanti, che il compilatore può piegare dentro al conto.



La taglia della rete entra nel programma **solo** nella P1.5, che passa da 776 a 1718 istruzioni fra 10 e 100 nodi.

Congelando il nodo quella dipendenza **scompare**: la riga della specializzata è piatta, fra 647 e 663 a qualunque taglia.

(La scala è logaritmica: in lineare le due righe P1 resterebbero schiacciate contro le quindicimila di P2.)

E la velocità

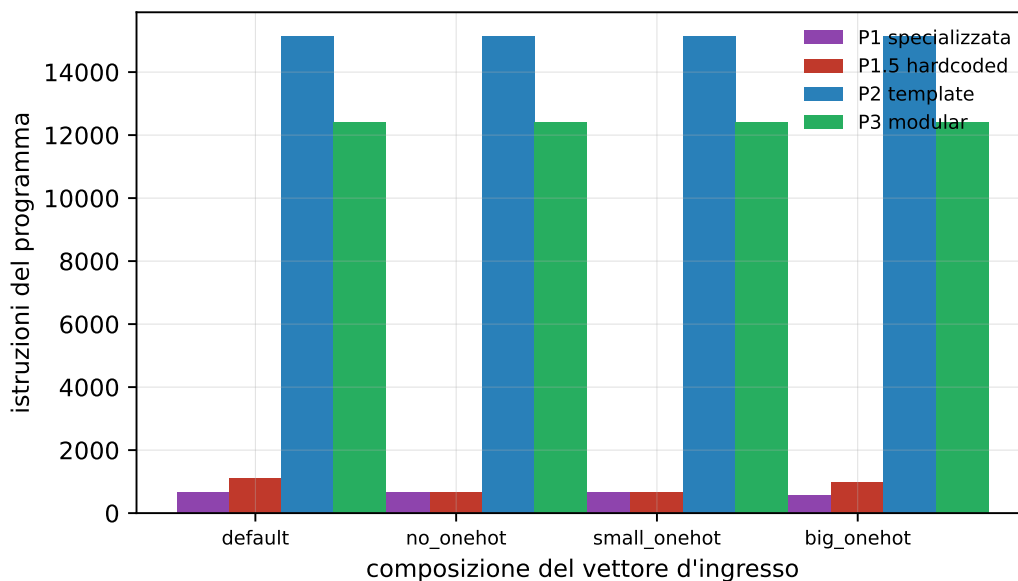
nodi	10	25	52	75	100
P1 specializzata	41	40	41	41	38 ns
P1.5 hardcoded	48	45	46	47	45 ns

La specializzata è più veloce di circa 6 ns, il 13%, a ogni taglia: la lettura della tabella del nodo e lo switch costano poco, ma si vedono. Entrambe sono piatte: a runtime lo switch è un salto indicizzato, qualunque sia il numero dei casi.

Il motivo per cui la *dimensione* crolla e il *tempo* cala poco è lo stesso: lo switch ha 52 casi ma ne esegue uno. Quelle 208 assegnazioni sono compilate per eseguirne quattro. Toglierle libera molto **spazio** e un po' di **tempo**.

Il controllo, e perché ci si può fidare

Come si fa a essere sicuri che il divario venga davvero dalla one-hot del nodo e non da qualcos'altro? Si prende un vettore d'ingresso che la feature *non* la contiene:



Nelle due composizioni senza la feature node le due P1 sono **identiche alla cifra** (656 e 656; 649 e 649). In quelle che ce l'hanno, divergono. Il divario è tutto lì e nient'altro.

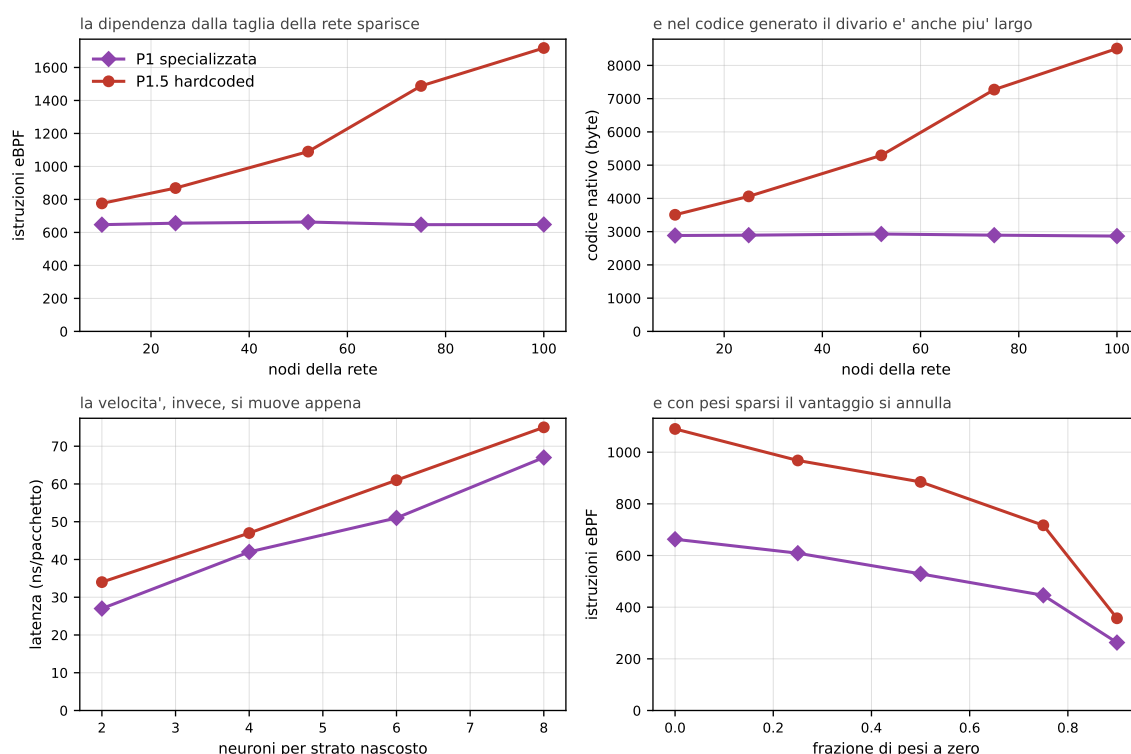
C'è poi una verifica che viene prima di ogni numero. Congelare il nodo significa scegliere *una colonna* della matrice dei pesi al momento di generare il codice: sceglierne una sbagliata darebbe un programma più piccolo, più veloce, e che calcola **un altro modello**. Ottanta casi, dieci valori di TTL per otto configurazioni di link: la versione congelata decide sempre la stessa classe di quella che legge il nodo dalla tabella.

I due vantaggi non si sommano

pesi a zero	0%	25%	50%	75%	90%
P1 specializzata	663	609	529	446	263
P1.5 hardcoded	1 090	968	885	717	357

Al 90% di zeri il divario scende da 427 a 94 istruzioni. Sparsità e nodo congelato sono **due strade alla stessa riduzione**: se i pesi sono già quasi tutti zero, il compilatore cancella lo switch da solo e non resta niente da congelare.

Le quattro risposte in una figura



Qui ci sono solo le due P1: in tutte le altre figure P2 e P3 stanno un ordine di grandezza più su e schiacciano questo confronto sul fondo del grafico. La dimensione crolla e smette di dipendere dalla rete; il codice nativo crolla ancora di più (2867 byte contro 8507 a cento nodi); la velocità migliora di poco; e con i pesi sparsi le due curve si toccano.

Il verdetto

Congelare il nodo **non è soprattutto un'ottimizzazione di velocità**. È ciò che rende la pipeline hardcoded *praticabile su una rete grande*.

Lo switch della P1.5 cresce col numero di nodi moltiplicato per la larghezza del primo strato. Su una topologia da 500 nodi con 8 neuroni sarebbero quattromila assegnazioni srotolate, e c'è una taglia oltre la quale quel programma non si carica più. La specializzata non ci arriva mai.

Il prezzo è **un binario per nodo**: 52 compilazioni (circa 70 ms ciascuna) e 52 installazioni.

12.7 Congelare quali porte esistono

La sezione precedente congela *dove* il nodo si trova. Questa congela *quante* interfacce ha.

Il modello riserva sei colonne per `link_state` perché sei è il grado del nodo più connesso della rete. Ma su un nodo di grado 3 le tre colonne restanti non sono link caduti: dietro non c'è alcuna interfaccia. Valgono zero adesso, varranno zero sempre.

Non si congela il **valore** della feature, si congela la sua **struttura**. Se un link sia su o giù resta letto a runtime; quello che diventa noto alla compilazione è *quali posizioni possono esistere*.

Il generatore non emette le colonne che restano fuori. Il termine che sparisce è `link_state[i] · w[j][i]` con `link_state[i]` nullo: la somma è fra interi a 64 bit, quindi togliere un addendo nullo non cambia un bit.

porte presenti	2	3	4	5	6
istruzioni eBPF	579	599	615	639	663
latenza	37	38	39	39	41 ns

21 istruzioni per porta, in modo lineare, e **4 ns** fra grado 6 e grado 2, con la latenza che sale in modo monotono. Diversamente dal nodo, qui le moltiplicazioni tolte vengono eseguite davvero a ogni pacchetto, e la misura le vede: circa un nanosecondo per porta.

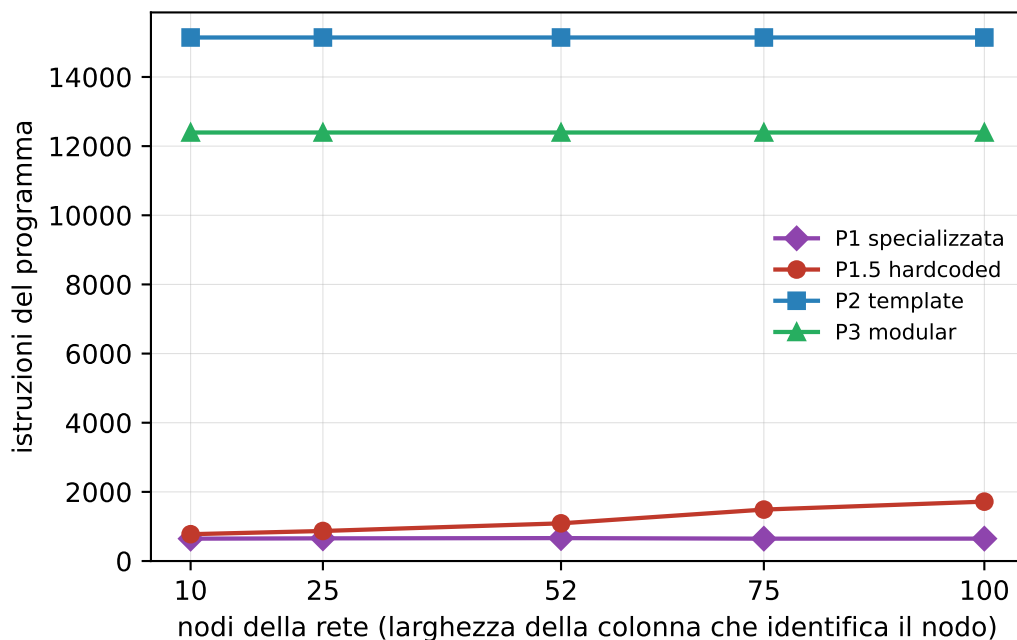
Le altre tre pipeline, che non specializzano niente, restano ferme lungo l'asse (1 090, 15 142 e 12 394 istruzioni, latenze piatte): la pendenza della specializzata viene dalle porte e da nient'altro.

Il controllo, che è servito davvero

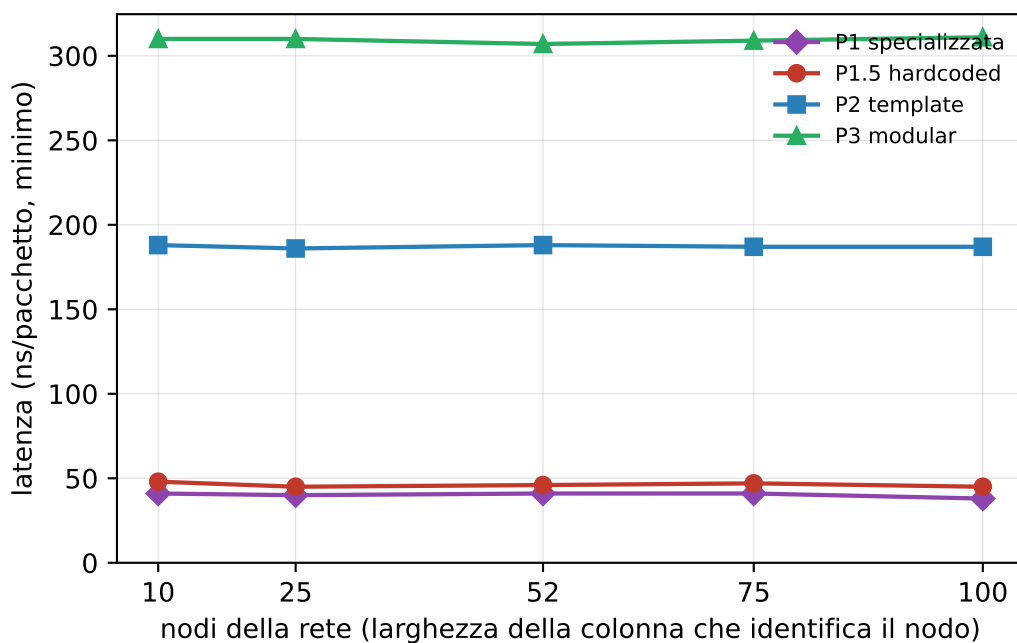
Ottanta casi — dieci valori di TTL per otto configurazioni di link realizzabili — danno la stessa classe fra la versione specializzata e quella a larghezza piena. Ma quel numero, da solo, non dimostra niente: se quelle colonne non spostassero mai la decisione, «le due versioni concordano» sarebbe vero anche per una specializzazione **sbagliata**. Serve un controllo al contrario: accendere uno slot che sul nodo non esiste, e pretendere che a quel punto le due divergano. Su alcuni pesi quel controllo resta muto — una colonna contribuisce $1 · w$ a un accumulatore che ne somma 65 — e il banco cerca fra più semi finché non morde: 29 divergenze.

Un test che passa anche quando la sua ipotesi è violata non sta verificando l'ipotesi.

12.8 Due larghezze, due risposte opposte

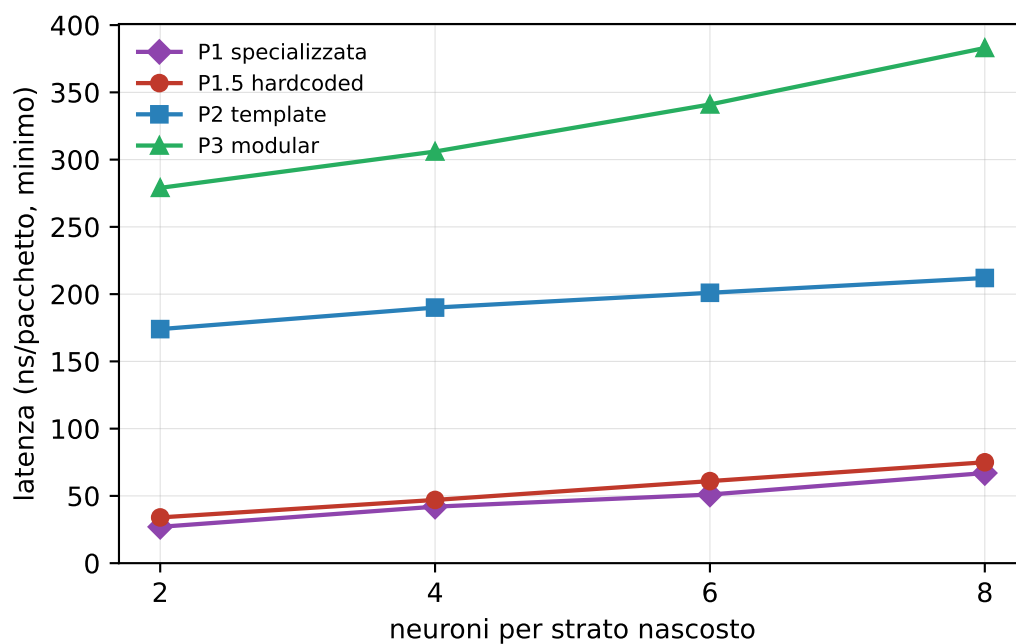


La taglia della *rete* entra nel programma solo in P1.5: è lo switch della one-hot che si srotola. P2 e P3 restano alla cifra esatta a ogni punto.



Piatte tutte. Cento nodi costano quanto dieci: una one-hot ha un solo uno, quindi il datapath legge *una sola colonna di pesi* qualunque sia la larghezza del vettore.

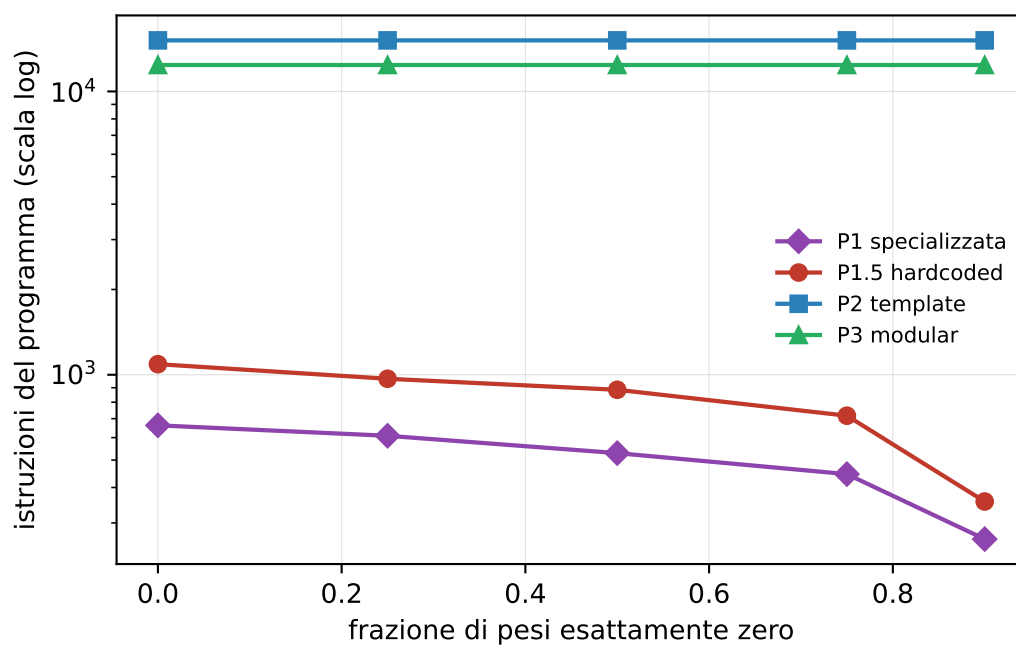
Allargare i layer *nascosti*, invece:



Da 2 a 8 neuroni: P1 specializzata 27 → 67 ns, P1.5 34 → 75, P2 174 → 212, P3 279 → 383.

La rete può crescere quanto vuole; il modello no.

12.9 I pesi contano, ma solo per una delle tre



Con il 90% dei pesi a zero, P1.5 passa da 1 090 a **357 istruzioni** e da 47 a 26 ns. P2 e P3 non si muovono di una istruzione, e di un paio di nanosecondi al massimo.

P1 scrive i pesi nel codice come numeri, quindi il compilatore vede una moltiplicazione per zero e la cancella. Per P2 e P3 uno zero è un byte in una tabella come un altro. È l'unico asse su cui dimensione e tempo scendono insieme.

12.10 Che cosa questi grafici non dicono

- Sono tutti BPF_PROG_TEST_RUN: il programma, non il nodo sotto traffico (capitolo 14).
- Oltre circa 115 nodi non si va senza alzare un tetto compilato e **rimisurare**.
- Che i *pesi*, a forma fissa, possano decidere se P1 si carica è possibile (il codice generato dipende dai valori), ma variando il solo seme dei pesi non è stato misurato. Con la ReLU senza bivio il verificatore ha molto margine (capitolo 10), quindi è meno probabile di prima.

Il costo del programma, misurato da solo

Prima di mettere un programma in una rete conviene sapere quanto costa da solo. Qui lo strumento è `BPF_PROG_TEST_RUN`: il kernel esegue il programma su un pacchetto preparato, in un ciclo, senza schede di rete, code o altri programmi di mezzo. È come cronometrare un cassiere mentre batte sempre lo stesso scontrino: misura il gesto, non la fila. La fila è l'argomento del capitolo 14.

13.1 La campagna sulle architetture

Per rispondere a *come cresce il costo quando il modello cresce* servono più modelli misurati con la stessa metodologia: quattro assi, un solo parametro che si muove per asse, minimo su sette prove.

Asse	Valori	Che cosa muove
<code>iv_dense</code>	n_{in} 5, 9, 13, 17	colonne d'ingresso tutte <i>dense</i> : ognuna è letta da una mappa e moltiplicata
<code>iv_onehot</code>	n_{in} 16, 32, 65	colonne d'ingresso in una <i>one-hot</i>
<code>width_camp</code>	65- $v-v-7$, v 4, 8, 16, 32	neuroni per strato nascosto
<code>depth_camp</code>	65-8...8-7, 1-4 strati	profondità, a larghezza fissa 8

I muri, tutti dichiarati

Larghezza 16 e 32 su Pipeline 2 e 3. I soffitti compilati valgono 8. E Pipeline 3 non rifiuta il caricamento: risponde `XDP_PASS` a runtime. Una cella a larghezza 16 *si caricherebbe, girerebbe, e restituirebbe una latenza* — di un programma che non calcola niente. Il banco controlla i soffitti *prima* di compilare e segna la cella RIFIUTATO.

Larghezza 32 su Pipeline 1. La compilazione con clang fallisce (lo stack di eBPF non basta). Ogni cella gira nel proprio sottoprocesso: la cella esce CRASH e lo sweep continua. Fino al 27 settembre anche larghezza 16 veniva rifiutata, dal verificatore, e P2 oltre due strati: con la ReLU senza bivio (capitolo 10) P1 a larghezza 16 carica (166–170 ns) e P2 carica tutti i punti di `depth_camp`.

13.2 Il problema: le MAC nominali non predicono la latenza

Il modo naturale di mettere in relazione complessità e tempo è contare le moltiplicazioni-accumulo dalla forma del modello: $n_{in} \times h_1$ per il primo strato, e così via. Fatto così, non funziona: con le MAC nominali sull'asse delle ascisse la retta spiega il 62% della varianza per la P1 specializzata, il 22% per P2 e l'8% per P3.

Il sintomo più chiaro sta sull'asse `iv_onehot`, dove il vettore d'ingresso quadruplica:

n_{in}	pesi	p1_static	hardcoded	template	modular
16	271	65 ns	65 ns	170 ns	363 ns
32	399	65 ns	64 ns	170 ns	367 ns
65	663	64 ns	64 ns	169 ns	367 ns

n_{in} per quattro, pesi per 2,4, e la latenza **non si muove su nessuna delle quattro**. Le istruzioni di Pipeline 1 intanto salgono da 1144 a 1632: la taglia statica cresce, il percorso eseguito no.

13.3 La soluzione: contare le MAC che vengono eseguite

La causa è nel datapath, ed è una riga sola. Il ramo che tratta una feature one-hot è questo:

```

} else if (code == FEAT_INGRESS_IF) {
    if (_raw_iface >= 1 && _raw_iface <= sz) {
        __u32 c = coff + (_raw_iface - 1);
        for (int j = 0; j < ML1_MAX_H1; j++)
            out[j] += LW_W(LW, woff + j * n_in + c);
    }
}

```

Una one-hot occupa sz colonne nella matrice dei pesi, ma nel datapath ne attiva *una*: il costo è h_1 addizioni, e non dipende da sz . Contarla come $sz \times h_1$ moltiplicazioni sovrastima, e di quanto dipende da quanta one-hot c'è nell'ingresso — cioè cambia da asse ad asse.

Il banco calcola quindi due colonne: *macs*, nominali, e *macs_eff*, che conta le one-hot per quello che eseguono. Con la seconda sull'asse delle ascisse:

Pipeline	MAC eseguite		MAC nominali	
	ns/MAC	r^2	ns/MAC	r^2
p1_static	0,287	0,99	0,070	0,62
hardcoded	0,290	0,98	0,076	0,70
template	0,548	0,71	0,085	0,22
modular	1,175	0,91	0,098	0,08

Per Pipeline 3 si passa da $r^2 = 0,08$ a $r^2 = 0,91$, e p1_static da 0,62 a 0,99. Quattro assi costruiti in modi diversi collassano sulla stessa retta, una per pipeline. P2 si ferma a $r^2 = 0,71$: ha un costo fisso grande rispetto alla pendenza, e il suo residuo più grande sta sull'asse della larghezza, dove i neuroni oltre la larghezza del modello vengono calcolati comunque fino al soffitto compilato.

Il costo per MAC eseguita è una proprietà misurabile della pipeline: circa 0,29 ns quando i pesi sono compilati come letterali, circa 1,18 ns quando vengono letti da una tabella (P3). Un fattore 4,1×: il prezzo della genericità espresso in una costante invece che in un aneddoto.

Il pavimento della baseline, che di MAC ne esegue zero, sta fra 18 e 23 ns: è il costo del framework XDP che ogni pipeline paga prima di moltiplicare qualunque cosa.

13.4 Due banchi, lo stesso programma, lo stesso numero

I due banchi a BPF_PROG_TEST_RUN misurano lo stesso programma, e devono dare lo stesso numero. Sul modello 65-4-4-7:

Pipeline	test_suite	campagna	scarto
baseline	18 ns	18 ns	0%
hardcoded	52 ns	46 ns	−12%
template	195 ns	190 ns	−3%
modular	315 ns	305 ns	−3%

Concordano entro qualche nanosecondo, e i programmi non sono identici: `test_suite` usa i pesi veri del modello (1 064 istruzioni per P1), la campagna pesi sintetici a sparsità nulla (1 090).

Il dettaglio che lo rende vero: un pacchetto nuovo ogni 200 giri

Il kernel, dentro `BPF_PROG_TEST_RUN`, non ripristina il pacchetto fra una ripetizione e l'altra. Il datapath decrementa il TTL: riusando lo stesso pacchetto, dopo una quarantina di esecuzioni il TTL resterebbe inchiodato a 1, e da lì in poi ogni ripetizione prenderebbe il ramo

```
if (ip->ttl <= 1) { /* ... */ return XDP_PASS; }
```

che sta **dopo** l'inferenza — il modello girerebbe comunque, e le curve scalerebbero in modo regolare — ma salta la coda di inoltrare: decremento, checksum, contatori, `mac_table`, `bpf_redirect`. Tutti i banchi rinfrescano quindi il pacchetto ogni 200 esecuzioni partendo da TTL 255 ($255 - 200 = 55$): nessuna ripetizione arriva alla scadenza.

Un difetto che lascia il modello girare è il più difficile da vedere: le curve restano belle. Si vede solo confrontando due banchi che devono dare lo stesso numero.

13.5 Il programma sul percorso vero: tre marcature

Con traffico vero si può cronometrare il programma *dentro* il percorso: il dispatcher marca T_1 all'ingresso, il programma marca T_2 subito prima di inoltrare, il contatore all'altro capo marca T_3 .

	baseline	P1	P1.5	P2	P3
$T_2 - T_1$ (il programma)	34 ns	68	77	228	340
$T_3 - T_2$ (redirect, veth, ricezione)	194–239	195–244	195–251	200–257	207–251

$T_2 - T_1$ è piatto sul carico, da 0,5 a 3 milioni di pacchetti al secondo; $T_3 - T_2$ è un costo comune di circa 195–255 ns che non dipende dalla pipeline. Sopra la baseline il solo programma costa +34 / +43 / +194 / +306 ns. La versione strumentata aggiunge due letture dell'orologio per pacchetto: se qualcosa, questi numeri sono gonfiati.

Il test reale: quanto costa un pacchetto al nodo

14.1 La domanda, in parole semplici

Un nodo della rete riceve pacchetti, per ognuno esegue la rete neurale, decide da quale porta farlo uscire, e lo spedisce. La domanda è: **quanti pacchetti al secondo regge il nodo, e quanto tempo spende su ognuno?**

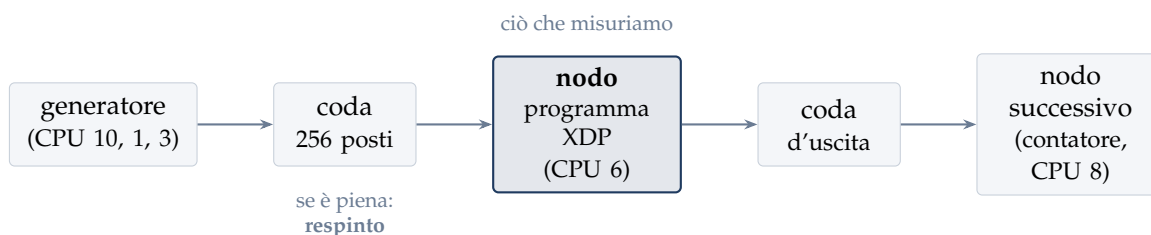
Un'immagine per tutto il capitolo: la cassa del supermercato

Il nodo è un **cassiere**. I pacchetti sono i **clienti**. Il generatore di traffico è la porta da cui entrano i clienti, e il collegamento virtuale fra generatore e nodo è il **nastro** su cui i clienti aspettano: ha 256 posti, e quando è pieno chi arriva viene mandato via.

Per sapere quanto spende il cassiere per cliente si fanno arrivare più clienti di quanti ne riesca a servire, così che il nastro non sia mai vuoto, e si conta quanti ne serve in un secondo. Se ne serve 5 milioni al secondo, spende $1/5\,000\,000$ di secondo a cliente, cioè 200 nanosecondi.

14.2 Com'è fatto il banco

Tutto sta sulla macchina di laboratorio, e ogni parte ha il suo core fisico, a frequenza fissa (capitolo 15):



- Il **generatore** (pktgen, nel kernel) crea pacchetti uguali a quelli veri: stesso formato, stesso header IPA, stesso modello da eseguire. Tre thread, tutti sulla stessa coda del nodo.
- La **coda d'ingresso** è quella di un *veth*, il cavo virtuale di Linux: 256 posti, e la dimensione non si può cambiare.
- Il **nodo** è una delle pipeline caricata in XDP. È l'unica cosa che si vuole misurare.
- Il **nodo successivo** conta i pacchetti arrivati e li butta.

Oltre alle cinque pipeline il banco misura, nella stessa sessione, *rxonly*: un programma che prende il pacchetto dalla coda e lo butta senza guardarlo. Dice quanto costa *soltanto ricevere*: è il pavimento sotto il quale nessuna pipeline può scendere.

14.3 Come si misura, e perché così

Un numero affidabile richiede cinque accortezze. Nessuna riguarda le pipeline: senza, tutte farebbero sembrare il nodo diverso da com'è.

1. La finestra stazionaria

`pktgen` si avvia con un comando che *blocca* finché la trasmissione non è finita, e il blocco non dura quanto la trasmissione: aspetta sempre 125 millisecondi fissi prima di partire, i suoi thread si accorgono del via con fino a 100 ms di ritardo, e alla fine controlla ogni 100 ms se hanno finito. Dividere i pacchetti per la durata del blocco vorrebbe dire contare fino a circa **225 ms di tempo morto** su finestre di misura da 300 ms.

Il banco aspetta che tutti i thread del generatore stiano trasmettendo, lascia assestare la coda, legge i contatori, aspetta 300 ms, li rilegge, e solo allora ferma il generatore. Il rate è la differenza fra le due letture divisa per il tempo fra le due letture.

2. Letture brevi

Ogni lettura (generatore più nodo) deve durare meno di 3 ms; se dura di più viene rifatta, e le si attribuisce l'istante a metà. Un processo interrotto a metà lettura attribuirebbe ai contatori un istante sbagliato di tutta l'interruzione. Un test senza kernel lo verifica con un generatore simulato.

3. I respinti contano

Quando la coda d'ingresso è piena il collegamento rifiuta il pacchetto («respinto»). Un pacchetto respinto è un pacchetto che il nodo non ha preso, perché non svuotava la coda abbastanza in fretta: è **una perdita del nodo**. Il banco conta separatamente i **respinti** (mai entrati) e i **persi dopo l'ingresso** (entrati ma non usciti).

4. La copia del generatore

`pktgen` crea i pacchetti come li crea lo stack di rete di Linux, in una *skb* con 64 byte liberi davanti; XDP ne vuole 256. Per ogni pacchetto il collegamento virtuale lo **copia** prima di darlo al nodo, e la copia la paga il processore del nodo. Su una scheda di rete con XDP nativo non esiste. Per toglierla c'è un secondo generatore (`ipa/test/xdp_gen.py`), che crea pacchetti già grezzi con `BPF_PROG_TEST_RUN` in modalità *live frames*.

5. Un core per il nodo successivo, una coda per il nodo

Il contatore d'uscita gira su un core suo, così che sul core del nodo resti solo il lavoro del nodo. E il nodo ha **una sola coda d'ingresso**, servita da un solo thread: con una coda per ogni thread del generatore il nodo avrebbe più thread sullo stesso core, e quale coda viene servita lo deciderebbe lo scheduler — a pieno carico ognuno terrebbe il core per millisecondi mentre le altre code traboccano.

14.4 I due generatori

Il banco ha due modi di produrre traffico, e servono tutti e due.

pktgen: il traffico come lo produce Linux

pktgen è un modulo del kernel che fabbrica pacchetti come li fabbrica lo stack di rete: ognuno è una *skb*, la struttura con cui Linux descrive un pacchetto, con 64 byte liberi davanti ai dati. XDP però ne vuole 256, quindi il cavo virtuale, prima di consegnare il pacchetto al nodo, lo **ricopia** in una pagina nuova. È un lavoro che il nodo paga, e che su una scheda di rete con XDP nativo non esisterebbe.

Tre thread su tre core offrono circa 5,7 milioni di pacchetti al secondo; la coda del nodo ne accetta circa 4,6 (quanto regge *rxonly*, il programma che riceve e butta).

xdp_gen: pacchetti già grezzi

Il secondo generatore (`ipa/test/xdp_gen.py`) consegna al cavo virtuale pacchetti già nel formato di XDP, con `BPF_PROG_TEST_RUN` in modalità *live frames*: niente *skb*, niente copia. Arriva a circa 7 milioni di pacchetti al secondo.

Esempio: lo stesso programma con i due generatori

La baseline — ricevi, leggi l'intestazione, inoltra — costa **302 ns** per pacchetto con pktgen e **141 ns** con xdp_gen. Il programma è lo stesso byte per byte: i 161 ns di differenza sono la copia della *skb* e il percorso dello stack, cioè il costo del *generatore*, pagato sul core del nodo. Per questo i due generatori non si confrontano fra loro, ma ognuno confronta le pipeline *al suo interno*: sopra la baseline, P2 costa 226 ns con il primo e 155 ns con il secondo, e P3 364 contro 293. La differenza fra le due stime è il costo del nodo che dipende da come arriva il pacchetto.

14.5 Com'era il test nella macchina virtuale, e com'è adesso

Fino al 26 settembre questi banchi giravano in una macchina virtuale VirtualBox a quattro core virtuali. Dal 27 girano su Ubuntu installato direttamente sul portatile, in dual boot. Il programma misurato è lo stesso; è cambiato tutto quello che gli sta intorno.

Il problema della macchina virtuale: il cassiere che sparisce

Un core virtuale non è un core: è un thread del sistema che ospita la macchina virtuale, e quel sistema può fermarlo quando vuole, anche per qualche millisecondo, per far girare altro. Dentro la macchina virtuale non si vede e non si può impedire.

Per il banco è fatale, e il conto è semplice. La coda d'ingresso del nodo ha 256 posti. A 1,5 milioni di pacchetti al secondo un pacchetto arriva ogni $1/1,5 \cdot 10^6 \approx 0,67$ microsecondi, quindi la coda si riempie in

$$256 \times 0,67 \mu s \approx 170 \mu s.$$

Se il core del nodo sparisce per più di 170 microsecondi — e un core virtuale lo fa — la coda trabocca e i pacchetti vengono respinti, *per quanto veloce sia il programma*. Nell'immagine del supermercato: il cassiere è bravissimo, ma ogni tanto esce dalla cassa per qualche secondo, e intanto il nastro si riempie e i clienti vengono mandati via.

Nella macchina virtuale questo dava due strade, entrambe cattive:

- **Generatore e nodo su core separati:** il nodo perdeva fra l'8 e il 70% dei pacchetti *a qualunque ritmo*, anche lentissimo. La perdita misurava le assenze del core, non il programma.

- **Generatore e nodo sullo stesso core:** la coda si svuota subito e non si perde niente, ma il core fa il lavoro di tutti e due. Il ritmo misurato è $1/(t_{\text{generatore}} + t_{\text{programma}})$, e il collo di bottiglia è sempre il generatore.

Si usava la seconda, sottraendo la baseline per togliere il costo del generatore. Il risultato, su quattro esecuzioni identiche nell'arco di mezz'ora:

sopra la baseline	macchina virtuale (18 settembre)		dual boot (28 settembre)	
	costo	variazione	costo	variazione
P1 statica	~82 ns	160%	38 ns	0,5%
P1.5	~116 ns	104%	53 ns	0,6%
P2	~430 ns	19%	226 ns	0,5%
P3	~670 ns	15%	364 ns	0,3%

Macchina virtuale: generatore e nodo sullo stesso core, 512 byte, variazione = (massimo – minimo)/media su quattro esecuzioni. Dual boot: core separati, 64 byte, variazione fra i giri della stessa sessione.

Le cifre assolute delle due colonne **non si confrontano**: nella macchina virtuale la frequenza del processore era quella che capitava, e il metodo era diverso. Si confronta la **variazione**. Per P1 e P1.5 era più grande del valore stesso: una volta P1 costava 164 ns, un'altra 32. Il loro ordine si invertiva da un'esecuzione all'altra, e l'unica cosa onesta da scrivere era “stanno sotto i 200 ns e non si distinguono”. Oggi variano meno dell'1%, e la differenza fra le due — 15 ns — si misura.

Che cosa ha cambiato il dual boot

- **Core veri, ognuno col suo compito.** Generatore sui core 10, 1 e 3, nodo sul core 6, nodo successivo sull'8. Nessun altro processo, nessuna interruzione e nessun thread del kernel sul core del nodo (capitolo 15).
- **Il core del nodo non dorme mai.** Gli stati di sonno profondi sono spenti: il risveglio dal più profondo costa 310 microsecondi, più dei 170 che la coda concede. Nella macchina virtuale anche sotto capacità si respingeva l'1–2% dei pacchetti; oggi **zero**.
- **Frequenza fissa e misurata:** 3 494–3 495 MHz sul core del nodo durante ogni misura di traffico, letta dai contatori del processore.
- **Si possono separare i core senza perdere**, quindi il generatore non è più dentro la misura: il ritmo a cui il nodo satura è quello del nodo.

Esempio: lo stesso punto misurato tre volte

Nella macchina virtuale, il banco del bit rate ha misurato lo stesso punto — stessa pipeline, stesso ritmo — in tre giri diversi, e ha perso il **58%**, lo **0,2%** e il **7,7%** dei pacchetti. Tre risposte diverse alla stessa domanda: nessuna era una misura del programma. Sul dual boot, cinque giri allo stesso punto danno lo stesso inoltro massimo entro lo 0,4–2%.

14.6 I limiti del test, con esempi

Il dual boot toglie il rumore. Non toglie i limiti di un banco che sta tutto su un portatile: vanno conosciuti, perché ognuno dice quali numeri si possono citare e quali no.

1. Il generatore ha un tetto

Se il nodo è più veloce di quanto il generatore riesca a produrre, il banco misura il generatore. `pktgen` arriva a circa 4,6 milioni di pacchetti al secondo consegnati, `xdp_gen` a circa 7.

Esempio: tre pipeline appiccicate al soffitto

Con `xdp_gen`, `rxonly` — che riceve e butta, e quindi fa meno di tutti — arriva a 6,89 milioni al secondo; la baseline a 7,11; P1 a 6,34; P1.5 a 6,23. Sono tutti lì, contro il soffitto dei 7 milioni: che la baseline superi `rxonly` dice proprio che nessuno dei due è il vero limite. P2 (3,38) e P3 (2,30) invece stanno ben sotto: loro si saturano, e il loro numero è il loro costo. Per questo la tabella dei pacchetti grezzi cita come costi solo P2 e P3.

Con `pktgen` il tetto è più basso ma le pipeline sono più lente (c'è la copia), e tutte e cinque saturano sotto di esso: è per questo che la tabella principale del capitolo viene da `pktgen`.

2. Contare all'uscita sottostima il nodo

Il nodo successivo, che conta i pacchetti arrivati, ha un core solo, e anche lui ha un tetto: circa 4,4–4,9 milioni al secondo con `xdp_gen`. Se si misura il nodo contando *ciò che arriva dall'altra parte*, quando il nodo è più veloce dell'uscita si misura l'uscita.

Esempio: la baseline che sembrava lenta

Con `xdp_gen` la baseline *elabora* 7,1 milioni di pacchetti al secondo, ma all'altra parte ne arrivano 4,4. P1 ne elabora 6,3 e ne arrivano 4,8. Contando gli arrivi, la baseline sembra più lenta di P1 — impossibile, perché fa strettamente meno lavoro. Più pacchetti la baseline spinge verso l'uscita, più l'uscita, già satura, ne perde.

Fino al 28 settembre il controllo di validità del banco (“la baseline dev’essere la più veloce”) contava proprio gli arrivi, e su ogni misura con `xdp_gen` dichiarava il run non valido. L’errore era del controllo: ora conta i pacchetti *elaborati dal nodo*, la stessa misura con cui il banco calcola il costo, e su tutti i run passa.

3. Il cavo è virtuale, e tutto sta su un processore

Il “ricevere” misurato è prendere il pacchetto dalla coda di un cavo virtuale, non dalla memoria di una scheda di rete; “inoltrare” è accodarlo sul cavo successivo, non scrivere un descrittore di trasmissione. Generatore, nodo e nodo successivo sono core dello stesso chip e condividono cache e memoria. Il portatile non ha una scheda di rete cablata che supporti XDP nativo.

Esempio: 217 ns per pacchetto di sola ricezione con `pktgen` e 145 con `xdp_gen` sono il prezzo *di questo percorso*. Su una scheda vera sarebbero altri numeri, e le cifre assolute di questo capitolo con loro. I **confronti** fra pipeline no: il percorso è lo stesso per tutte, e si sottrae.

4. Un solo tipo di pacchetto per volta

In una misura tutti i pacchetti sono uguali: stesso modello, stesso TTL, stessa classe decisa. Il processore impara in fretta un flusso così (il predittore dei salti, la cache), e un traffico

misto costerebbe un po' di più. Le classi si misurano una alla volta, ed è così che si è visto che scartare costa più che inoltrare.

5. Le latenze si leggono solo al minimo

Il tempo di attraversamento si raccoglie in un istogramma a potenze di due: i percentili sono il bordo superiore della casella, con risoluzione di un fattore 2. Esempio: la mediana a basso carico è "1 024 ns" per baseline, P1 e P1.5 e "2 048 ns" per P2 e P3 — due caselle, non due misure. Il **minimo** invece è esatto (249, 282, 292, 460, 584 ns), ed è l'unico che si cita.

6. Il punto esatto in cui comincia la perdita è instabile

Il banco cerca anche il ritmo più alto a perdita nulla, restringendo per tentativi. Quella ricerca oscilla: fra un giro e l'altro dà valori diversi fino al 64%, perché al confine basta un'esitazione di pochi microsecondi per perdere un pacchetto. La saturazione invece varia fra lo 0,2 e il 2%. Si citano quindi la saturazione e il carico comune, non il confine; il bit rate lo descrive per gradini (sezione seguente).

7. Il portatile scalda

Oltre 3,5 GHz non regge una misura intera, e anche a 3,5 capita che il processore nel suo insieme tocchi per un attimo la soglia di temperatura, di solito mentre clang compila sugli altri core. Il banco lo registra: il 28 settembre è successo 3 volte durante la misura della latenza e mai durante quelle di throughput e di bit rate, e il core del nodo è rimasto a 3 495 MHz per tutto il tempo.

14.7 Il risultato: quanto regge il nodo

Con pktgen, pacchetti da 64 byte, tre giri per tre ripetizioni:

Pipeline	milioni di pacchetti/s	variazione fra i giri	sopra la baseline
solo ricezione (rxonly)	4,61	0,2%	—
baseline (nessuna rete)	3,31	0,9%	—
P1 statica	2,94	0,5%	+38 ns
P1.5 (hardcoded)	2,81	0,6%	+53 ns
P2 (template)	1,89	0,5%	+226 ns
P3 (modular)	1,50	0,3%	+364 ns

La **baseline** riceve, legge l'intestazione, decrementa il TTL e inoltra. Tutto ciò che le pipeline aggiungono sopra è il **costo della rete neurale**: circa 40–50 ns quando i pesi sono scritti nel programma (P1, P1.5), circa 220 ns quando la forma della rete è letta da una tabella (P2), circa 370 ns quando anche la profondità lo è (P3).

Come si legge la colonna di destra, con un esempio. La baseline regge 3,31 milioni di pacchetti al secondo, cioè spende $10^9 / 3,31 \cdot 10^6 = 302$ ns per pacchetto; P2 ne regge 1,89 milioni, cioè 529 ns. La differenza, con le cifre non arrotondate $528,6 - 302,5 = 226$ ns, è il tempo che P2 spende a eseguire la rete neurale: tutto il resto — ricevere, leggere l'intestazione, inoltrare — lo fanno tutte e due allo stesso modo.

E sotto la capacità? Offrendo a tutte lo stesso carico — 1,37 milioni di pacchetti al secondo, il 91% di quanto regge la più lenta — **nessuna pipeline respinge un solo pacchetto**,

e fra ingresso e uscita se ne perdono al massimo lo 0,02%. Un nodo sotto la sua capacità consegna tutto.

P1 e P1.5 si separano

2,94 contro 2,81 milioni al secondo, con variazioni fra i giri dello 0,5 e dello 0,6%: congelare il nodo vale circa 15 nanosecondi sul percorso vero (340 contro 356 ns per pacchetto). Piccolo, ma fuori dal rumore. Lo stesso ordine lo danno il banco del bit rate (2,94 contro 2,84) e il cronometro del solo programma (68 contro 77 ns).

La ReLU senza bivio, sul traffico vero

Nel banco a pacchetto ripetuto P1 ha perso circa 5 ns con la ReLU senza bivio (capitolo 10). Qui no: sopra la baseline P1 costava 38 ns il 27 settembre e costa 38 ns oggi, P1.5 53 e 53. Il motivo è il predittore dei salti del processore: quando lo stesso pacchetto passa milioni di volte di fila impara da che parte va ogni bivio, e un bivio indovinato costa quasi niente — per questo toglierlo, nel banco, sembrava un costo. Sul percorso vero il resto del lavoro (ricevere, copiare, inoltrare) pesa molto di più, e la differenza sparisce nel rumore.

14.8 Il costo del nodo, con pacchetti grezzi

Con il generatore a pacchetti grezzi (xdp_gen, sezione 14.4) e il nodo successivo su un core suo, il costo per pacchetto sul core del nodo — dalla presa del pacchetto alla consegna all'uscita — è:

Pipeline	ns per pacchetto
P2 (template)	296–307
P3 (modular)	433–444
baseline, P1, P1.5	141, 158, 161

al tetto del generatore: limiti superiori

5 giri a 64 byte, e 3 giri per taglia a 64, 512 e 1 514 byte.

Solo P2 e P3 sono costi del nodo. Le tre più leggere elaborano 6,2–7,1 milioni di pacchetti al secondo, e il generatore a pacchetti grezzi non ne produce più di circa 7: il loro numero è quello del generatore, e dice solo che il nodo costa *al massimo* tanto. La sezione sui limiti (14.6) lo mostra con i numeri.

- **La taglia del pacchetto non conta:** a 64, 512 e 1 514 byte il costo è lo stesso entro il 3%. Il nodo guarda solo l'intestazione.
- **La porta d'uscita non conta:** le cinque classi che inoltrano costano uguale entro l'11%.
- **Scartare costa di più che inoltrare:** la classe DROP costa 260 ns su P1 contro 158, 330 su P2, 468 su P3. Quando il pacchetto viene scartato la sua pagina di memoria si restituisce sul core del nodo; quando viene inoltrato, su quello del nodo successivo. È un effetto del banco, non della pipeline.

14.9 Al crescere del bit rate

La domanda del relatore: al crescere del traffico offerto la coda si riempie, il tempo di attraversamento cresce, e poi si perde. Se si perde *all'ingresso* del programma, il collo di bottiglia è il programma; se si perde *dopo*, è la trasmissione. Il banco del bit rate mette davanti a ogni pipeline un secondo programma che conta soltanto (una catena di due programmi

sullo stesso aggancio, con un salto), e misura tredici valori di traffico calibrati sul tetto di ricezione della macchina, cinque giri ciascuno.

Pipeline	pulita fino a	perde da	inoltro massimo
baseline	1,44 Gbit/s	1,68 Gbit/s	3,34 M/s
P1 statica	1,20	1,44	2,94
P1.5	1,20	1,44	2,84
P2	0,72	0,96	1,89
P3	0,72	0,96	1,50

pacchetti da 64 byte; bit rate misurato sul collegamento; 28 settembre

Le prime due colonne sono **gradini**, non soglie esatte. I tredici valori sono spaziati di circa 0,24 Gbit/s, e “pulita fino a 0,72, perde da 0,96” vuol dire che la perdita comincia da qualche parte fra i due. Esempio: P2 inoltra al massimo 1,89 milioni di pacchetti al secondo, cioè $1,89 \cdot 10^6 \times 64 \times 8 \approx 0,97$ Gbit/s; il gradino da 0,96 cade proprio sul suo limite, e perde. P3 (1,50 milioni, $\approx 0,77$ Gbit/s) perde allo stesso gradino, pur essendo più lenta: fra 0,72 e 0,96 non c’è un valore intermedio che le separi. A distinguerle è la terza colonna.

Per tutte e cinque la perdita è **all’ingresso**: il collo di bottiglia è il programma eBPF. Dopo il programma si perde al massimo lo 0,02% di ciò che è stato inviato. E il tempo di attraversamento fa quello che la domanda prevedeva: 9–10 microsecondi a basso carico, fino a 69–194 al primo valore in cui si perde, cioè la coda d’ingresso piena.

Il secondo programma della catena costa poco, e si misura invece di supporlo: il kernel tiene il tempo di ogni programma attaccato, e con e senza il contatore davanti la differenza va da -2 a $+5$ ns, entro la variazione fra le ripetizioni della misura stessa.

14.10 Che cosa questo numero non dice

I limiti del banco sono nella sezione 14.6. Uno in più riguarda il modello:

- **Un modello solo.** Il costo sul traffico vero è misurato sul modello depositato (65-4-4-7); come cresce con modelli più grandi lo dicono i capitoli precedenti, a livello di programma.

La macchina di laboratorio, sotto controllo (27 settembre)

L'ultimo giorno di settembre i banchi sono passati sulla macchina di laboratorio vera, senza strati di mezzo: un portatile Lenovo IdeaPad Slim 5 con un **Intel Core Ultra 7 155H**, Ubuntu 24.04, kernel 6.8. Tutte le misure di questo quaderno sono state rifatte lì, lo stesso giorno. Questo capitolo racconta che cosa è servito per poterle rifare bene.

15.1 Un processore che non sta fermo

Il 155H è **ibrido**: tre tipi di core diversi sullo stesso chip.

tipo	CPU	frequenza	note
P-core (prestazioni)	0–11	base 1,4, turbo 4,5–4,8 GHz	6 core fisici, due thread ciascuno fuori dalla cache condivisa
E-core (efficienza)	12–19	base 0,9, turbo 3,8 GHz	
LP E-core	20–21	base 0,7, turbo 2,5 GHz	

Lasciato a sé stesso, un banco su questa macchina finisce dove capita. La regola che il banco usava per scegliere i processori (un terzo al nodo, il resto al generatore) qui dava quattordici thread di generatore e metteva il nodo sui core di efficienza. Il desktop e il browser giravano sul gemello del core che si stava misurando. La frequenza andava da 0,4 a 4,8 GHz secondo il carico, e i core si addormentavano in uno stato da cui il risveglio costa 310 microsecondi: più di quanto la coda d'ingresso da 256 posti copra a un milione di pacchetti al secondo.

E il portatile **scalda**. Un solo P-core salito a 4,5–4,8 GHz porta il processore da 51 a 99 °C in due secondi, e il chip riduce la frequenza per difendersi (*throttling*): dopo 41 minuti di uso leggero, due core contavano già migliaia di questi interventi.

15.2 Le condizioni, messe e tolte

`ipa/test/host_conditions.py` mette la macchina in condizioni note per la durata di un banco, e le riporta com'erano alla fine.

- **Ruoli**: nodo, nodo successivo e generatore su P-core fisici *diversi*, un thread per core; il gemello di ciascuno resta a riposo; il core della CPU 0 resta al sistema. Qui: nodo sulla CPU 6, nodo successivo sull'8, generatore su 10, 1 e 3.
- **Frequenza fissa** sui core del banco (governor *performance*, minimo uguale al massimo), e **misurata** dai contatori del processore (APERF/MPERF).
- **Tetto sui core del sistema** (2,5 GHz): il turbo del desktop scalderebbe il chip fino a rallentare anche i core del banco.
- **Niente sonno profondo** sui core del banco: spenti gli stati con risveglio oltre un microsecondo.

- **Isolamento:** desktop, servizi, interruzioni e thread del kernel spostabili confinati sulle CPU del sistema.
- **Frequenza della cache condivisa fissa**, profilo energetico *performance*, watchdog spento.

Ogni valore si salva *prima* di cambiarlo, in un file in memoria. A fine run — anche con Ctrl-C o un errore — si riscrive tutto all'indietro; un run ucciso lascia il file, e il run successivo ripristina per primo. Un test sul kernel vero (`test_host_kernel.py`, 25 controlli) applica le condizioni, le rilegge dal kernel, verifica che un processo del desktop giri solo sulle CPU del sistema, misura la frequenza sotto carico, ripristina e confronta la macchina valore per valore con com'era — anche dopo aver ucciso il processo a condizioni applicate.

Durante ogni misura un **monitor** legge gli interventi di throttling, la temperatura, la frequenza vera del nodo e le interruzioni del firmware. Una finestra misurata fuori dalle condizioni dichiarate viene marcata e contata, non scartata in silenzio.

15.3 La frequenza scritta non è la frequenza vera

La prima scelta era la frequenza *base*, 1,4 GHz, l'unica garantita. Il sistema la accettava e la rileggeva: 1 400 minimo, 1 400 massimo. I contatori del processore dicevano un'altra cosa:

Con 1 400 MHz scritti nel sistema, il core gira a **circa 2 000 MHz** — fisso e stabile, ma non a quella frequenza. Da 3 000 MHz in su scritto e vero coincidono entro 10 MHz.

Se il monitor si fosse fidato del numero scritto, avrebbe marcato come disturbate 38 finestre su 39. Da allora la frequenza di riferimento è quella **misurata** all'avvio, su ogni core del banco; il numero scritto resta accanto, per confronto.

15.4 Quale frequenza

Più alta la frequenza, più alto il throughput — finché il portatile regge. La risposta si è misurata:

frequenza vera	rxonly	baseline	P2	macchina
2,0 GHz	2,65	2,00	1,14	pulita
3,0 GHz	3,93	2,89	1,66	pulita
3,5 GHz	4,37	3,22	1,86	pulita
4,0 GHz	4,84	3,55	2,09	pulita in 30 s; in 5 minuti fino a 98 °C
4,5 GHz	5,17	3,54	2,04	throttling su 28 finestre su 39

milioni di pacchetti al secondo inoltrati, 64 byte

A 4,5 GHz il throughput smette di crescere: il chip si difende. A 4 GHz un run corto è pulito, uno da cinque minuti porta il processore a 98 °C e il core del nodo stesso in throttling. **3,5 GHz** è la frequenza più alta che la macchina regge per un run intero: 1 finestra disturbata su 390, massimo 85 °C. È diventata la frequenza di default del banco.

La frequenza pesa quasi in proporzione, ma non del tutto: fra 2 e 4 GHz i cicli per pacchetto crescono dell'8–13%, perché la cache condivisa e la memoria non accelerano con il core. Quello che non si muove è il **rapporto fra pipeline**: P2 regge il 57–59% della baseline a ogni frequenza. Le cifre assolute si citano con la loro frequenza; i rapporti no.

15.5 Il risultato: misure che si ripetono

banco	variazione
programma da solo (test_suite, fra sette prove)	2–5%
throughput a saturazione (fra tre giri)	0,1–1,7%
inoltro massimo al crescere del bit rate (fra cinque giri)	0,4–2,0%

E i pacchetti respinti con il nodo sotto capacità, che prima erano l'1–2% anche per il programma che non fa niente, sono **zero**: l'unica causa rimasta era il risveglio del core, e il core non dorme più.

15.6 Prima e dopo: che cosa è cambiato

Fino al 26 settembre i banchi giravano in una macchina virtuale VirtualBox a quattro core virtuali, sullo stesso portatile. Dentro non si vedeva la frequenza, non si sceglieva il core fisico, e il sistema sotto poteva fermare un core virtuale per millisecondi. Il 27 settembre tutto è passato su Ubuntu installato direttamente sul portatile (dual boot), con le condizioni di questo capitolo. Confrontando le misure di prima con quelle di oggi ("oggi" è il 28 settembre, con la ReLU senza bivio del capitolo 10; il dettaglio sul test con il generatore è nel capitolo 14):

Che cosa è migliorato

	prima	oggi
programma da solo, latenza (baseline / P1 / P2 / P3)	28 / 76 / 311 / 445 ns	18 / 52 / 195 / 315 ns
dispersione fra le prove	13–30%	2–6%
costo della rete su traffico vero, sopra la baseline	P2 ~430, P3 ~670 ns; P1 e P1.5 non misurabili	38 / 53 / 226 / 364 ns
variazione fra i run	15–19% per P2 e P3, oltre il 100% per P1 e P1.5	0,2–0,9% fra i giri
respinti con il nodo sotto capacità	1–2% anche per il solo contatore	zero
bit rate: lo stesso punto in giri diversi	58% / 0,2% / 7,7% di perdita	inoltro massimo entro lo 0,4–2%
P1 contro P1.5	indistinguibili, l'ordine si inverte fra i run	2,94 contro 2,81 M/s, sempre nello stesso ordine
P3, costo per strato in più	~70 ns	~53 ns
frequenza del core	né visibile né controllabile	fissa e misurata: 3 494 MHz

Il guadagno vero non è la velocità, è la **ripetibilità**: la dispersione scende di un ordine di grandezza. Differenze che prima sparivano nel rumore — P1 contro P1.5, una porta in più o in meno, scartare contro inoltrare — ora si misurano. Le latenze assolute di prima e di oggi invece non si confrontano: prima la frequenza era quella che capitava.

Che cosa è peggiorato, o è diventato un limite

- **P2 oltre due strati nascosti non si caricava più.** Nella macchina virtuale si caricava fino a sei strati (da 14 668 a 18 801 istruzioni); qui il verificatore la rifiutava da tre strati in

su. La causa non era il kernel ma il codice: ogni ReLU era un salto condizionale, e già a uno strato il verificatore percorreva 800 000 istruzioni su un limite di un milione. Con la ReLU senza salto (28 settembre) P2 carica fino a sei strati.

- **P1 a ~1 200 pesi, lo stesso.** Sull'ingresso di IPA prima si caricavano tutte le forme, dalla larga alla profonda (122–293 ns); qui passava solo quella a otto strati. Stessa causa di P2, stessa correzione: con la ReLU senza salto il tier B carica in tutte le forme.
- **Il portatile scalda.** Oltre 3,5 GHz non regge un run intero, quindi il banco non può usare il turbo.
- **Serve più preparazione:** la frequenza scritta nel sistema non è quella vera, clang non trova un header senza un percorso in più, e sul sistema nuovo PyTorch non è installato, quindi la parte userspace di `test_suite.py` non parte finché non lo si installa.

Nessuna misura è peggiorata. Sono peggiorati due **limiti di caricamento**, e sembravano dipendere dal kernel. Le statistiche del verificatore (`diag_verifier.py`) hanno detto altro: i programmi lavoravano già vicino al limite di complessità, per colpa di un salto condizionale in ogni ReLU, e una piccola differenza fra kernel bastava a superarlo. Tolto il salto, il margine torna ampio (P2 a sei strati usa il 14% del limite).

15.7 Che cosa ha portato alla luce

Rifare tutto su una macchina ferma ha trovato cose che il rumore nascondeva:

- **P2 oltre due strati nascosti non si caricava**, e **P1 oltre qualche centinaio di pesi** nemmeno: la causa comune, un bivio in ogni ReLU, è nel capitolo 10.
- **Le porte assenti** costano un nanosecondo ciascuna, e ora si vede; **congelare il nodo** vale il 16% di latenza, e ora si vede.
- **Scartare costa più che inoltrare** sul banco a pacchetti grezzi.
- Tre difetti del banco: con il compilatore di sistema i programmi di P1 non trovavano un header (manca una cartella nel percorso di ricerca, ora aggiunta); il banco del throughput scriveva i risultati nella cartella sbagliata; e il generatore scriveva su una coda per thread, lasciando allo scheduler la scelta di quale servire (ora una coda sola, sezione 5 del capitolo 14).

15.8 Come si rifà tutto

Due script rifanno ogni misura di questo quaderno, nelle stesse condizioni. I CSV vanno in `results/`, i registri in `/tmp/ipa_logs`, e i grafici del quaderno si rigenerano dai CSV in `docs/figures/`:

```
bash ipa/test/remeasure_all.sh      # programma, correttezza, analisi parametrica
bash ipa/test/remeasure_traffic.sh  # traffico vero, semantica, soffitti, topologie
```

Le altre correzioni di settembre

Il test reale non è stata l'unica cosa sistemata nell'ultima settimana. Questo capitolo racconta le altre, una sezione ciascuna: che cosa non andava, perché contava, che cosa è stato fatto, e come si è verificato che fosse a posto.

16.1 Il P1 che va sui nodi non inoltrava

Il contesto. P1 si può costruire in due modi. Con *BCC*, che compila il programma sul nodo al momento del caricamento (serve un compilatore sul nodo). Oppure *AOT* (*ahead of time*): si compila prima, su un'altra macchina, e al nodo si spedisce l'oggetto già pronto, che un piccolo caricatore in C apre e attacca. È la versione AOT quella che va sui nodi.

Il difetto. Il caricatore AOT riempiva la tabella delle uscite con l'interfaccia numero 1 per tutte le porte: *loopback*, il collegamento interno della macchina con sé stessa. Ogni pacchetto da inoltrare veniva perso. Nessun test se ne accorgeva, perché i test di consegna provavano la versione BCC.

La correzione. Il caricatore lascia le tabelle vuote e le «appunta» nel file system del kernel; il pannello di controllo in Python le riempie con le interfacce vere del nodo, poi dice al caricatore di attaccare il programma. Il caricatore stacca tutto e pulisce quando riceve lo stop.

La prova. Il test di consegna ha una variante che usa esattamente il percorso di deploy: **11 casi su 11**, con ogni classe consegnata sull'interfaccia giusta e lo scarto contato.

16.2 Tutti i test di P1 usano l'oggetto AOT

Ogni test e ogni banco che tocca P1 o P1.5 usa l'oggetto AOT, attraverso un modulo solo (`ipa/p1_aot.py`): la prova di consegna, le prove sul kernel, i modelli sintetici, l'analisi parametrica, i banchi su traffico vero.

Prova	esito
consegna su rete di veth (tutte le pipeline)	29/29
consegna della P1 come si deploia	11/11
cinque topologie diverse	34/34
equivalenza numerica sui modelli sintetici, P1	8/8 scenari, 300/300 ingressi ciascuno
P1 statica contro P1.5: stessa decisione	80/80 casi
semantica delle classi per modello, P2 e P3	53/53

16.3 Il cronometro del caricatore AOT

Il difetto. Il caricatore misurava eseguendo il programma un milione di volte *sullo stesso pacchetto*, e il kernel non rimette a posto il pacchetto fra un'esecuzione e l'altra: dopo poche decine di giri il TTL scadeva, e si misurava il ramo corto «pacchetto scaduto», non l'inoltro. È lo stesso difetto raccontato per il banco parametrico (capitolo 13).

La correzione. Pacchetti nuovi ogni 200 esecuzioni, partendo da TTL 255. Il risultato: 989 istruzioni, 47 ns per pacchetto sul percorso d'inoltro, la stessa cifra di `test_suite`.

16.4 Due implementazioni che devono dare lo stesso numero

Il rischio. Il progetto ha un *riferimento*: il calcolo della rete scritto in Python, con gli stessi numeri interi del kernel. Se il riferimento e il codice generato per il kernel sbagliano *allo stesso modo*, ogni test passa.

Il difetto trovato. Il riferimento usato per i modelli sintetici non moltiplicava il bias per la scala giusta (il bias dello strato l va moltiplicato per s^l). Sul modello depositato non si vedeva.

La correzione. La scala è un argomento obbligatorio del riferimento, e ogni file di risposte attese dichiara lo schema e la scala con cui è stato calcolato. In più un controllo confronta direttamente **il riferimento Python con il codice C generato per P1**, valutato come testo, logit per logit. Per provare che il controllo sa fallire, lo si è fatto girare su codice alterato apposta: se ne accorge.

16.5 La scala delle feature in P2 e P3

Il TTL entra nella rete diviso per una costante scelta durante l'addestramento, e la costante viene dal modello. La prova *nel kernel* per P2 e P3 ha un controllo negativo: su un modello addestrato con scala 16, se si altera la scala scritta nella tabella del nodo le decisioni cambiano esattamente nei 41 casi su 300 previsti (89 nel modello `small`); con la scala giusta tornano identiche al riferimento. Quindi la scala viene davvero letta e usata.

P2 accetta ora modelli con un numero di classi d'uscita diverso da quello configurato: il numero viene dalla semantica dichiarata del modello.

16.6 Pulizia del repository

Tolto ciò che non corrispondeva più al codice o all'ambiente di oggi:

- gli script per inviare e ricevere pacchetti a mano fra i nodi del vecchio laboratorio (`send_ipa.py`, `recv_ipa.py`, `test_ipa.py`): senza quei nodi non avevano più un destinatario, e la consegna la verifica `test_fabric.py`;
- la sonda dell'ambiente (`probe_env.sh`), sostituita da `host_conditions.py -show` e `test_host_kernel.py`;
- un controllo sui sorgenti C che nessun test eseguiva più;
- da `results/` tutto ciò che non è una misura: restano solo i CSV. Grafici, registri e resoconti si rigenerano dai CSV; i grafici di questo quaderno stanno in `docs/figures/`.

Tenuti, dopo averli controllati, due script che sembravano superati: `diag_p3_bisect.py` (rifà la misura dei tetti del verificatore) e `bench_tailcall_overhead.py` (misura un costo che nessun altro isola).

I numeri di oggi, in una pagina

	baseline	P1 statica	P1.5	P2	P3
<i>Il nodo su traffico vero (capitolo 14), 64 byte</i>					
milioni di pacchetti al secondo	3,31	2,94	2,81	1,89	1,50
costo della rete (sopra la baseline)	—	38 ns	53 ns	226 ns	364 ns
pulita fino a (bit rate)	1,44	1,20	1,20	0,72	0,72 Gbit/s
<i>Il solo programma</i>					
$T_2 - T_1$ sul percorso vero (mini-mo)	34 ns	68 ns	77 ns	228 ns	340 ns
BPF_PROG_TEST_RUN, pesi del modello	18 ns	—	52 ns	195 ns	315 ns
istruzioni del programma	155	663	1 064	15 142	12 394
<i>Cambiare modello su un nodo in servizio</i>					
caricare sul nodo	—	0,7–2,1 ms (AOT)		10–12 ms	
anche generare e compilare	—	66–120 ms		non serve	
serve un compilatore sul nodo?	—	no (compilato prima)		no	

Tutto misurato il 28 settembre, con la ReLU senza bivio (capitolo 10), core del banco a 3,5 GHz. P1 statica ha i pesi sintetici dell'analisi parametrica per le istruzioni. Il bit rate si misura su una griglia di valori calibrata sulla sessione: "pulita fino a" è l'ultimo punto della griglia senza perdite, non la capacità esatta.

In tre righe:

1. **Scrivere i pesi nel programma costa 40–50 ns per pacchetto; leggerli da una tabella 220–370 ns.** È il prezzo della configurabilità.
2. **Con l'oggetto precompilato, cambiare modello costa pochi millisecondi per tutte le pipeline.** Il vantaggio residuo di P2 e P3 è non dover compilare niente, da nessuna parte.
3. **Congelare anche il nodo (P1 statica) vale poco ma si vede:** circa 40 ns sul percorso vero, 7 ns sul solo programma, e la dimensione non dipende più dalla taglia della rete.

Quel che resta

18.1 Fatto

- semantica delle classi dichiarata, mai dedotta; catena classe → azione → porta logica → interfaccia;
- motore indipendente dallo scenario, dimostrato su cinque topologie diverse;
- verifica end-to-end su datapath reale con XDP native;
- errore del bias nell'ultimo strato corretto;
- feature dell'interfaccia d'ingresso resa viva in tutte e tre le pipeline;
- **indice del nodo** letto a runtime da tabella in tutte e tre: pesa 208 pesi su 319;
- primo strato riscritto con i cicli invertiti in P2 e P3: stesso risultato bit per bit, e il margine di verifica che ne è uscito ha permesso di **recuperare il tetto sulle code** (da 1 a 8) e di **contare le letture di tabella**;
- oggetto precompilato (AOT) come unico deploy di P1, e come unica P1 misurata;
- **analisi parametrica** su sette assi e quattro pipeline, e **campagna** sulle architetture: le pendenze sono diverse, ed è la misura che giustifica l'esistenza di più pipeline;
- **versione P1 specializzata**, per il nodo e per le porte presenti;
- **semantica delle classi per modello** in P2 e P3 (capitolo 3);
- **banco su traffico vero** con generatore, nodo e nodo successivo su core separati, e un controllo di validità che il banco esegue su sé stesso;
- **banco del bit rate**: dove comincia la perdita, e da che parte;
- **la macchina sotto controllo** (capitolo 15): misure ripetibili entro pochi punti, frequenza vera misurata, niente respinti sotto capacità, e tutto rifatto con due script;
- **ReLU senza bivio** in P1, P2 e P3 (capitolo 10): P2 carica fino a venti strati invece di due, P1 carica il tier da ~1 200 pesi.

18.2 Aperto

- **Rimisurare le tre pipeline con la ReLU senza bivio** (capitolo 10): istruzioni e latenze cambiano di poco, e sugli assi della profondità e del budget di pesi compaiono punti che prima non caricavano.
- **Il costo del nodo delle pipeline leggere** con pacchetti grezzi: il banco non le satura. Servono più thread di generatore e più di un core per il nodo successivo.
- **Architetture diverse su traffico vero**: il costo del nodo è misurato su un modello solo.
- **Hardware vero**: una scheda di rete con XDP nativo e una seconda macchina, per un numero che non sia di un cavo virtuale.

- **Il tetto sull'uscita di P2:** lo strato finale è srotolato a 32 classi per un modello che ne ha 7. Abbassarlo ridurrebbe il programma ma *restringe l'insieme dei modelli rappresentabili*.
- **I salti fra programmi di P3:** tre per pacchetto, ed è lì (con le letture che li accompagnano) che sta la latenza.
- **Isolamento dal boot:** i core del banco sono isolati a sistema acceso; l'isolamento completo (tick e callback del kernel) richiede parametri di avvio.

18.3 Il filo conduttore

Guardando indietro, quasi tutti i problemi trovati a settembre hanno la stessa forma. Non erano calcoli sbagliati: erano **assunzioni non dichiarate** che risultavano vere per caso nell'unica configurazione mai provata.

Assunzione	Vera solo perché
classe k = interfaccia k	le porte erano numerate come le classi
DROP = ultima classe	nel modello depositato capitava così
$n_{\text{nodi}} = 52$ di default	si provava una topologia sola
eth0 = ifindex 2	si provava su un sistema appena avviato
bias dell'uscita $\times s^0$	si provava coi link tutti attivi
one-hot del nodo = model_id	si registrava un modello solo
tabella vuota = nodo 0	l'array è preallocato a zero
frequenza scritta = frequenza vera	non la si era mai misurata

E quasi tutte sono emerse allo stesso modo: cambiando *una* condizione che nessun test aveva mai cambiato.

Un test verde dice che il codice si comporta come il test si aspetta. Non dice che il test stia chiedendo qualcosa.

I comandi

Tutti i comandi si lanciano dalla radice del repository. `sudo` serve ovunque si carichi un programma nel kernel o si tocchi un'interfaccia; senza `sudo` girano solo le prove che non escono da Python.

Prerequisiti

Linux con BCC installato; per l'oggetto AOT di P1 anche

```
sudo apt install clang libbbpf-dev libelf-dev zlib1g-dev libzstd-dev liblzma-dev
```

Il generatore di traffico (`pktgen`) il banco lo carica da sé.

19.1 La macchina

```
# topologia, piano dei ruoli e condizioni attuali (niente root)
python3 ipa/test/host_conditions.py --show

# un comando qualunque sul core del nodo, a condizioni applicate
sudo python3 ipa/test/host_conditions.py --run -- CMD ...

# se un run e' stato ucciso: rimette la macchina com'era
sudo python3 ipa/test/host_conditions.py --restore

# le condizioni verificate: su un /sys finto, e sul kernel vero
python3 ipa/test/test_host_conditions.py
sudo python3 ipa/test/test_host_kernel.py --bench

# tutte le misure del quaderno
bash ipa/test/remeasure_all.sh
bash ipa/test/remeasure_traffic.sh
```

19.2 Correttezza: il modello fa quello che deve

```
python3 ipa/test/test_suite.py
python3 ipa/test/test_suite.py --only core --verbose
python3 ipa/test/test_class_semantics.py
python3 ipa/test/test_synth.py
python3 ipa/test/verify_synth_kernel.py --all --dry-run
python3 ipa/test/diag_model_decisions.py
```

19.3 Kernel: si carica, e quanto costa

```
sudo python3 ipa/test/host_conditions.py --run -- \
    python3 ipa/test/test_suite.py --only kernel
sudo python3 ipa/test/verify_multi_model.py
```

```
sudo python3 ipa/test/verify_per_model_semantics.py
sudo python3 ipa/test/verify_synth_kernel.py --all --n 300
sudo python3 ipa/test/verify_synth_kernel.py --all --n 300 --pipeline p2
sudo python3 ipa/test/verify_synth_kernel.py --all --n 300 --pipeline p3
```

19.4 Datapath reale: il pacchetto attraversa davvero

```
sudo python3 ipa/test/test_fabric.py
sudo python3 ipa/test/test_fabric.py --method aot
sudo python3 ipa/test/test_fabric.py --sweep
sudo python3 ipa/test/netns_fabric.py --cleanup
```

19.5 Analisi parametrica: come cresce il costo

```
sudo python3 ipa/test/host_conditions.py --run -- \
python3 ipa/test/bench_scaling.py --axis all --out results/
sudo python3 ipa/test/host_conditions.py --run -- \
python3 ipa/test/bench_scaling.py --axis campaign --out results/
python3 ipa/test/bench_scaling.py --plot results/
sudo python3 ipa/test/bench_scaling.py --verify
sudo python3 ipa/test/bench_scaling.py --verify-ports
sudo python3 ipa/test/host_conditions.py --run -- \
python3 ipa/test/bench_depth_vs_width.py
```

19.6 Il test reale

```
# throughput a saturazione e a carico comune (pktgen)
sudo python3 ipa/test/bench_throughput.py --mode compare --rounds 3 \
--out results/throughput_3500

# il bit rate crescente: latenza, ricevuti, inoltrati, dove si perde
sudo python3 ipa/test/bench_bitrate.py --out results/bitrate_3500
python3 ipa/test/bench_bitrate.py --report results/bitrate_3500

# le tre marcature T1/T2/T3 su una scala di rate
sudo python3 ipa/test/bench_throughput.py --mode rates --frames 64 \
--rounds 3 --rates 0.5,1,1.5,2,2.5,3

# pacchetti grezzi, nodo successivo su un core suo; per classe
sudo python3 ipa/test/bench_throughput.py --mode compare --generator xdp \
--egress-cpu auto --rounds 5
sudo python3 ipa/test/bench_throughput.py --mode compare --generator xdp \
--egress-cpu auto --per-class --rounds 3

# la finestra di misura e le formule, verificate senza kernel
python3 ipa/test/test_steady_window.py
python3 ipa/test/test_bitrate_math.py

# se un run si interrompe e lascia il generatore configurato
sudo python3 ipa/test/bench_throughput.py --cleanup
```

Come si legge l'uscita

Condizioni del test — in testa, e in `env.csv`: ruoli, frequenza scritta e misurata, temperatura, interventi di throttling durante il run.

Controllo di validità — la baseline deve essere la più veloce entro il 10% e `rxonly` non deve essere più lento della baseline. Se una delle due cose non vale, il run misura il banco e non le pipeline.

Finestre marcate — «macchina» (fuori dalle condizioni dichiarate), «raffica» o «gen» (il generatore non ha offerto il carico chiesto): contate e riportate, non scartate.

19.7 Costi specifici e oggetto precompilato

```
sudo python3 ipa/test/bench_tailcall_overhead.py
sudo python3 ipa/test/bench_model_add.py --n-models 3
python3 ipa/methods/method4_hardcoded_aot.py      # build, come utente
sudo python3 ipa/methods/method4_hardcoded_aot.py # bench
sudo python3 ipa/test/diag_p3_bisect.py --ceilings
```

19.8 L'ordine consigliato

Partendo da zero su una macchina nuova:

1. `python3 ipa/test/host_conditions.py -show` — com'è fatta la macchina, e dove andrebbe il banco?
2. `python3 ipa/test/test_suite.py` — il modello è sano?
3. `sudo python3 ipa/test/test_host_kernel.py` — le condizioni si mettono e si tolgono davvero?
4. `sudo python3 ipa/test/test_fabric.py` — il pacchetto attraversa davvero?
5. `bash ipa/test/remeasure_all.sh` e `bash ipa/test/remeasure_traffic.sh` — tutti i numeri.